

APELLIDOS: _____ NOMBRE: _____

Normas: Escribe los apellidos y el nombre en cada hoja de examen y en cada hoja adicional que utilices. Es posible escribir la respuesta en la misma hoja de cada ejercicio, pero si utilizas hojas adicionales no mezcles en la misma hoja respuestas de bloques diferentes: se entregará cada bloque por separado. La respuesta a los ejercicios debe estar escrita con bolígrafo. Está permitido el uso de calculadora aunque no es necesaria.

[0.75] 1. M1 y M2 son dos máquinas con el mismo repertorio de instrucciones y el mismo compilador. Hay 5 clases de instrucciones (A, B, C, D y E) dentro del repertorio de instrucciones. M1 tiene una frecuencia de reloj de 5 GHz y M2 tiene una frecuencia de reloj de 4.5 GHz. La siguiente tabla muestra el número medio de ciclos por instrucción para cada tipo de instrucción, así como su frecuencia de aparición en un programa dado:

Tipo de instrucción	CPI M1	CPI M2	Frecuencia (%)
A	1.0	0.8	10
B	2.5	2.0	15
C	1.2	1.0	30
D	3.0	2.5	25
E	2.0	1.8	20

Contesta, de forma razonada y mostrando todos los pasos del cálculo, a las siguientes preguntas:

(a) ¿Qué máquina es más rápida y por cuánto?

$$CPI\ M1 = \sum_i CPI_i \times F_i = 0.1 \times 1 + 0.15 \times 2.5 + 0.3 \times 1.2 + 0.25 \times 3 + 0.2 \times 2 = 1.985$$

$$CPI\ M2 = \sum_i CPI_i \times F_i = 0.1 \times 0.8 + 0.15 \times 2 + 0.3 \times 1 + 0.25 \times 2.5 + 0.2 \times 1.8 = 1.665$$

$$T_{CPU} = CPI \times N \times \frac{1}{f_{CPU}} \rightarrow A = \frac{CPI_{M1} \times N \times f_{M2}}{CPI_{M2} \times N \times f_{M1}}$$

La máquina M2 es $\frac{1.985 \times 4.5\ GHz}{1.665 \times 5\ GHz} = 1.073$ veces más rápida.

- (b) Si las instrucciones de tipo A y C son instrucciones de punto flotante, determina los GFLOPS obtenidos por la CPU M2 durante la ejecución del programa.

$$F_{M2} = 4.5 \text{ GHz}$$

$$CPI_{M2} = 1.67$$

$$GIPS_{M2} = \frac{4.5 \times 10^9 \text{ ciclos/s}}{1.67 \text{ ciclos/instr}} = 2.70 \text{ GIPS}$$

$$GFLOPS_{M2} = 40\% \times 2.70 \text{ GIPS} = 1.08 \text{ GFLOPS}$$

- (c) ¿Cuánto necesitaríamos mejorar el coprocesador de punto flotante de la máquina más lenta para que igualase el rendimiento de la más rápida?

Se necesita un factor de mejora de 1.073 sobre el CPI de M1.

El porcentaje del tiempo de ejecución empleado en computar flops:

$$CPI_{M1} = 1.985$$

$$CPI_{M1,flops} = 0.1 \times 1 + 0.3 \times 1.2 = 0.46$$

$$f_{M1,flops} = \frac{0.46}{1.985} = 23.2\%$$

Aplicamos Amdahl:

$$1.073 = \frac{1}{0.768 + \frac{0.232}{x}} \Rightarrow x = 1.415$$

Las operaciones en punto flotante deben ejecutarse 1.415 veces más rápido.

APELLIDOS: _____ NOMBRE: _____

- [2.0] 2. Considera un computador con un sistema de memoria virtual paginado con páginas de 4 KiB (2^{12}) y un esquema de traducción en 2 niveles. El espacio virtual es de 16 GiB (2^{34}). El espacio físico es de 64 MiB (2^{26}). Cada tabla de páginas de primer o segundo nivel ocupa exactamente una página física, y cada entrada se compone de un bit de validez, un bit de modificación y el número de página física. Dispone de una TLB de 4 entradas.

A continuación se muestra el contenido de la TLB, de parte de la tabla de páginas de nivel 1 (TP_1), residente en la página física 0x3000, y de una tabla de páginas de nivel 2 (TP_2). Todos los valores están en **hexadecimal**.

TLB		TP_1		TP_2	
PV	PF	Entrada	Contenido	Dirección	Contenido
004FC4	13A1	...	...	...	...
354043	0354	6A7	0354	[1BC0086]	8FC4
354044	2BC0	6A8	9BC0	[1BC0088]	ABC0
2BC004	23A1	6A9	9BC1	[1BC008A]	DFAB
		...	...	...	...

Contesta **razonadamente** las siguientes cuestiones:

- (a) [0.3] Indica cómo se divide y cuántos bits tiene cada campo de las direcciones virtuales y físicas.

Las DV (34 bits) tienen 22 bits de número de PV y 12 bits de desplazamiento. Como las tablas de primer y segundo nivel son iguales, el número de bits de PV1 y PV2 debe ser el mismo, por tanto, 11 bits de PV1, 11 bits de PV2 y 12 de Δ_p .

Las DF tienen 26 bits, 14 bits de número de página física y 12 de desplazamiento.

- (b) [0.7] Dadas las siguientes direcciones virtuales, 0x2BC004354 y 0x354045ADA, realiza el proceso de traducción a su correspondiente dirección física. En caso de necesitar tablas de primer y segundo nivel, considera que su contenido es el mostrado en el enunciado de este ejercicio. Ten en cuenta que para TP_1 se indica el **número de cada entrada** mientras que para TP_2 se indica la **dirección** de cada entrada.

- 0x2BC004354 se puede traducir en de la TLB: la dirección resultante es 0x23A1354.
- 0x354045ADA hay que traducirla a través de las tablas de páginas. En esa dirección, $PV_1 = 0x6A8$ y $PV_2 = 0x045$. TP_1 indica que $RBTP_2$ es 1BC0 tras comprobar que el bit de residencia es 1. La dirección de la entrada en TP_2 es $1BC0000 + 045 * 2 = 1BC008A$ y su contenido 0xDFAB. El bit de residencia es 1 y la PF 0x1FAB. La dirección resultante es 0x1FABADA.

- (c) [0.3] En la TLB y en TP_1 hay una entrada con valor 0x0354. Razona si es posible que ambas tengan **ese** mismo valor y sea correcto.

En la TLB es la página física que se corresponde con la PV 0x354043. En la Tabla de Páginas el bit de residencia está a 0, por lo que el resto de bits no son significativos. Por tanto, sí es correcto que coincida ese valor.

- (d) [0.3] El sistema cuenta con una caché de datos de primer nivel de 32 KiB. El tamaño de línea es 16 Bytes. Determina cuál debe ser su asociatividad para poder utilizar etiquetas físicas e índices virtuales.

Si la caché fuese de correspondencia directa, el índice y el desplazamiento ocuparían 15 bits, independientemente del tamaño de línea. Para que ocupen 12 bits el índice debe reducirse en al menos 3 bits, y por tanto la asociatividad debe ser como mínimo $2^3 = 8$.

- (e) [0.2] Si la traducción se hiciese en 3 niveles en lugar de 2, manteniendo el tamaño de página actual, ¿las tablas de páginas de un proceso necesitarían más o menos memoria física? Razona tu respuesta considerando el concepto de “fragmentación”.

Ahora mismo cada tabla ocupa una página física, que es lo mínimo que se puede reservar en memoria. Si la traducción se hace en 3 niveles, necesitaremos más tablas, más pequeñas, pero que en memoria seguirían ocupando lo mismo cada una (se desperdicia espacio en cada página reservada → fragmentación interna), por lo que en conjunto ocuparán más espacio.

- (f) [0.2] Si en el sistema descrito se ejecutan 4 procesos simultáneamente, indica qué cantidad de memoria virtual y de memoria física está disponible para cada uno de ellos.

Cada proceso tiene un espacio virtual independiente del resto, de 16 GiB. El espacio físico también está a su disposición completamente, pero se compartirá con el resto de procesos según las circunstancias lo requieran.

APELLIDOS: _____ NOMBRE: _____

[0.75] 3. Sea un computador con las siguientes características:

- Las direcciones de memoria y las palabras son de 64 bits.
- El sistema de memoria y de bus soportan transacciones de bloques de hasta 16 palabras.
- Tiene un bus síncrono de 128 bits a 100 MHz, en el que tanto una transferencia de 128 bits como el envío de una dirección de memoria requieren 1 ciclo de reloj.
- En una transacción el tiempo de acceso a memoria para las 4 primeras palabras es de 80 ns, mientras que para cada grupo adicional de 4 palabras es de 30 ns.
- Las transferencias por el bus y los accesos a memoria pueden solaparse.
- Entre transacciones no hay ciclos de espera.
- El sistema de memoria virtual utiliza páginas de 1 KiB.

Calcula la latencia y el ancho de banda para la lectura de una página.

$$T_{ciclo} = \frac{1}{f} = \frac{1}{100MHz} = 10^{-8} s = 10 ns$$

El sistema soporta bloques de 16 palabras. Por tanto, para transferir 1 KiB, se requiere: $1024 B / (16 \times 8 B/transacción) = 8$ transacciones.

Para leer las 16 palabras de 1 transacción necesitamos:

- 1 ciclo para enviar la dirección.
- Un tiempo de acceso a memoria de $(80 ns / (10 ns/ciclo)) = 8$ ciclos para leer las 4 primeras palabras.
- 2 ciclos para enviar los datos del grupo de 4 palabras leído (el bus es de 128 bits, por lo que enviamos 2 palabras por ciclo), que se solapa con el tiempo de acceso del siguiente grupo de 4 palabras que sería $30 ns / (10 ns/ciclo) = 3$ ciclos
 - Como cada uno de estos grupos está formado por 4 palabras, necesitamos repetir 4 veces para transferir las 16 palabras que forman 1 transacción.

Por tanto, para la lectura de 16 palabras de 1 transacción necesitamos $1 + 8 + 3 \times 3 + 2 = 20$ ciclos/transacción.

La latencia para la lectura de 1 KiB será: $8 \text{ transacciones} \times 20 \text{ ciclos/tr} \times 10 \text{ ns/ciclo} = 1600 \text{ ns} = 1.6 \mu s$

Y el ancho de banda será: $1 \text{ KiB} / (1.6 \times 10^{-6} s) = 64 \times 10^7 \text{ B/s} = 640 \text{ MB/s}$

[0.5] 4. Contesta las siguientes preguntas:

- (a) [0.3] Tenemos un RAID 1+0 formado por discos de 1 TiB con una capacidad de almacenamiento neta de 5 TiB y queremos reconfigurarlo a un RAID 6 aprovechando todos los discos. ¿Cuál sería la capacidad de almacenamiento neta de la nueva configuración? Razona la respuesta.

Un RAID 1+0 de 5 TiB de capacidad de almacenamiento neta necesita 10 discos de 1 TiB. El RAID 6 tendría una información redundante que ocuparía el espacio de 2 discos de 1 TiB, así que nos quedaría una capacidad de almacenamiento neta de 8 discos de 1 TiB, es decir, 8 TiB.

- (b) [0.2] Indica dos ventajas de los discos de estado sólido (SSD) frente a los discos duros de almacenamiento magnético (HDD).

Algunas ventajas que podemos enmuerar son las siguientes: tienen mayor velocidad de transferencia, menor latencia, tiempo de acceso uniforme, menor consumo energético, son silenciosos, se calientan menos, menor tamaño y peso, y mayor fiabilidad.

APELLIDOS: _____ NOMBRE: _____

Normas: Escribe los apellidos y el nombre en cada hoja de examen y en cada hoja adicional que utilices. Es posible escribir la respuesta en la misma hoja de cada ejercicio, pero si utilizas hojas adicionales no mezcles en la misma hoja respuestas de ejercicios diferentes: se entregará cada ejercicio por separado. La respuesta a los ejercicios debe estar escrita con bolígrafo. Está permitido el uso de calculadora aunque no es necesaria.

[0.75p] 1. M1 y M2 son dos máquinas con el mismo repertorio de instrucciones y el mismo compilador. Hay 4 clases de instrucciones (A, B, C y D) dentro del repertorio de instrucciones. M1 tiene una frecuencia de reloj de 500 MHz y M2 tiene una frecuencia de reloj de 750 MHz. El número medio de ciclos por instrucción para cada tipo de instrucción es el siguiente

Tipo de instrucción	CPI M1	CPI M2
A	1	2
B	2	2
C	3	4
D	4	4

(a) Si el número de instrucciones ejecutado en cierto programa se divide equitativamente entre los tipos de instrucciones, ¿cuánto más rápida es M2 respecto a M1?

$$Tiempo\ M1 = \frac{N \times CPI_{M1}}{F1} = N \times \frac{1 + 2 + 3 + 4}{4} \times \frac{1}{500 \times 10^6} = N \times 5\ ns$$
$$Tiempo\ M2 = \frac{N \times CPI_{M2}}{F2} = N \times \frac{2 + 2 + 4 + 4}{4} \times \frac{1}{750 \times 10^6} = N \times 4\ ns$$

La máquina M2 es $(5 \times N)/(4 \times N) = 1.25$ veces más rápida.

(b) ¿A qué frecuencia de reloj tendría M1 el mismo rendimiento que M2?

$$\frac{N \times CPI_{M1}}{F1} = \frac{N \times CPI_{M2}}{F2}$$
$$\frac{2.5}{F1} = \frac{3}{750 \times 10^6} \Rightarrow F1 = \frac{2.5}{3} \times 750 \times 10^6 = 625MHz$$

APELLIDOS: _____ NOMBRE: _____

- [2p] 2. Considera un computador con un sistema de memoria virtual paginado, una TLB de 8 entradas y una única caché de 2 KiB (2^{11} bytes) asociativa por conjuntos de 4 vías con un tamaño de línea de 16 bytes. El tamaño del espacio virtual es de 512 MiB (2^{29} bytes) y el tamaño de página es 256 bytes (2^8 bytes). Cada entrada de la tabla de páginas ocupa 2 bytes, y contiene un bit de residencia seguido del número de página física. No tiene bits de control adicionales.

Contesta **razonadamente** las siguientes cuestiones:

- (a) **0.2p** Determina el tamaño de la memoria principal.

En la tabla de páginas cada entrada tiene 2 bytes, de los cuales 15 bits son el número de PF. Entonces la memoria principal tiene 2^{15} páginas de 256 bytes cada una: $2^{15} \times 2^8 = 2^{23} B = 8 MiB$

- (b) **0.3p** Razona, a partir del tamaño de la tabla de páginas, si sería posible que el sistema utilizase traducción directa en un único nivel.

El espacio virtual tiene $2^{29}/2^8 = 2^{21}$ páginas virtuales. La tabla de páginas ocuparía $2^{21} \times 2 = 2^{22}$ bytes, es decir, la mitad de la memoria física disponible, lo cual, aunque es posible, no es práctico ya que el sistema sólo permitiría ejecutar 2 procesos simultáneamente y no quedaría memoria física disponible para sus datos.

- (c) **0.3p** Determina si es posible que la caché tenga índices virtuales y etiquetas físicas (VIPT). En caso negativo, indica qué tamaño, asociatividad o tamaño de línea debería tener la caché para que fuese así.

La caché tiene $2^{11-4-2} = 32$ conjuntos, por lo que el índice (5 bits) y el desplazamiento (4 bits) ocupan más que el desplazamiento de página (8 bits). Es necesario o bien reducir el tamaño de la caché a la mitad o duplicar la asociatividad para reducir el índice en 1 bit. El tamaño de línea no afecta.

- (d) **0.4p** Cada tabla de páginas ocupa exactamente 1 página física. Calcula cuántas entradas tiene cada tabla de páginas y en cuántos niveles se realiza el proceso de traducción.

Cada tabla tiene $2^8/2 = 2^7$ entradas. La memoria virtual tiene $2^{29}/2^8 = 2^{21}$ páginas. Como el número de página virtual tiene 21 bits, el proceso de traducción se realiza en $21/7 = 3$ niveles.

- (e) **0.3p** En la traducción de la dirección virtual 0x116453C, la entrada correspondiente de la tabla de páginas de último nivel (la última tabla consultada antes de terminar el proceso) contiene el valor 0xFF45. Calcula la dirección física resultante.

De los 16 bits de la entrada, el bit de residencia tiene valor 1, y a continuación la página física: 0x7F45. La dirección física es por tanto 0x7F453C.

- (f) **0.3p** En la traducción anterior, si la tabla de páginas de último nivel reside en la página física 0x4130, calcula la dirección de la entrada que debe ser consultada.

$0x116453C \rightarrow PV3 = 0x45$ (últimos 7 bits antes del desplazamiento). El registro base de la tabla es $RBTP_3 = 0x413000$. $Dir = 0x413000 + 0x45 \times 2 = 0x41308A$.

- (g) **0.2p** Tras esa traducción, indica qué información se almacenará en la TLB.

Se guardará el par PV-PF: 0x11645 - 0x7F45.

APELLIDOS: _____ NOMBRE: _____

[0.5] 3. Marca con una $\times$ la única respuesta correcta en cada apartado (cada respuesta correcta vale 0.1 y cada respuesta incorrecta resta 0.05).

- (a) ¿Cuál de las siguientes afirmaciones describe mejor un *array* de discos (RAID)?
- ☐ Es un sistema de almacenamiento que utiliza una única unidad de disco para mayor eficiencia.
 - ☐ Es un tipo de disco óptico utilizado principalmente para almacenar archivos multimedia.
 - ☒ Es un método para combinar múltiples discos en un solo volumen con el fin de mejorar la redundancia y/o el rendimiento.
- (b) ¿Cuál de las siguientes afirmaciones describe mejor el concepto de RAID 5?
- ☐ Utiliza una técnica de duplicación de datos para mejorar la redundancia.
 - ☒ Distribuye tiras de paridad a lo largo de todos los discos para eliminar cuellos de botella.
 - ☐ Emplea paridad y otro ECC para recuperarse de dos fallos de disco simultáneos.
- (c) ¿Qué tipo de dispositivos de almacenamiento se consideran como opciones de almacenamiento secundario de un sistema?
- ☒ Discos duros (HDD) y unidades de estado sólido (SSD).
 - ☐ Memoria caché y registros de la CPU.
 - ☐ Memoria RAM y ROM.
- (d) ¿Cuál es el nivel de RAID que permite recuperarse de dos fallos de disco simultáneos?
- ☐ RAID 0.
 - ☒ RAID 6.
 - ☐ RAID 5.
- (e) ¿Cuál de las siguientes ventajas de los discos de estado sólido (SSD) en comparación con los discos duros de almacenamiento magnético (HDD) **no es cierta**?
- ☒ Menor coste por GByte.
 - ☐ Mayor fiabilidad.
 - ☐ Mayor velocidad de transferencia.

[0.75] 4. Sea un computador con las siguientes características:

- Un sistema de memoria y de bus que soportan el acceso a bloques de 64 palabras de 64 bits cada una.
- Un bus síncrono de 128 bits a 4 GHz, en el que tanto una transferencia de 128 bits como el envío de una dirección de memoria requieren 1 ciclo de reloj.
- En una transacción el tiempo de acceso a memoria para las 8 primeras palabras es de 10 ns, mientras que para cada grupo adicional de 8 palabras es de 2 ns.
- Las transferencias por el bus y los accesos a memoria pueden solaparse. Se supone que no hay ciclos de espera entre transacciones.

Calcula la latencia y el ancho de banda para la lectura de 1024 palabras.

$$T_{ciclo} = \frac{1}{f} = \frac{1}{4GHz} = \frac{1}{4}10^{-9} s = 0.25 ns$$

El sistema soporta bloques de 64 palabras. Por tanto, para transferir 1024 palabras, se requieren $1024 p / (64 p/transacción) = 16$ transacciones.

Para leer las 64 palabras de 1 transacción necesitamos:

- 1 ciclo para enviar la dirección.
- Un tiempo de acceso a memoria de $10 ns / (0.25 ns/ciclo) = 40$ ciclos para leer las 8 primeras palabras.
- 4 ciclos para enviar los datos del grupo de 8 palabras leído (el bus es de 128 bits, por lo que enviamos 2 palabras por ciclo), que se solapa con el tiempo de acceso del siguiente grupo de 8 palabras que sería $2 ns / (0.25 ns/ciclo) = 8$ ciclos
 - Como cada uno de estos grupos está formado por 8 palabras, necesitamos repetir 8 veces para transferir las 64 palabras que forman 1 transacción.

Por tanto, para la lectura de las 64 palabras de 1 transacción necesitamos $1 + 40 + 7 \times 8 + 4 = 101$ ciclos/transacción

La latencia para la lectura de 1024 palabras será $16 transacciones \times 101 ciclos/tr \times 0.25 ns/ciclo = 404 ns$

Y el ancho de banda será $(1024 palabras \times 8 B/palabra) / (404 \times 10^{-9} s) = 20.28 \times 10^9 B/s = 22.28 GB/s$

APELLIDOS: _____ NOMBRE: _____

Hojas adicionales adjuntas: _____

Normas: Escribe los apellidos y el nombre en cada hoja de examen y en cada hoja adicional que utilices. Es posible escribir la respuesta en la misma hoja de cada ejercicio, pero si utilizas hojas adicionales no mezcles en la misma hoja respuestas de ejercicios diferentes: se entregará cada ejercicio por separado. La respuesta a los ejercicios debe estar escrita con bolígrafo. Está permitido el uso de calculadora. Al terminar, indica en cada hoja de examen las hojas adicionales utilizadas para cada ejercicio.

Un sistema cuenta con un procesador con una frecuencia de reloj de 4 GHz. El ancho de palabra es de 64 bits. La memoria principal es de 1 GiB (2^{30} B), divididos en 2 módulos de 512 MiB con entrelazamiento de orden inferior configurados en *Dual Channel*.

Dispone de memoria caché organizada en dos niveles

- L1: 32 kiB (2^{15} B), líneas de 64 Bytes, asociativa por conjuntos de 8 vías, con algoritmo de reemplazo LRU y políticas de ubicar en escritura (*write-allocate*) y post-escritura (*write-back*)
- L2: 256 kiB (2^{18} B), líneas de 64 Bytes, asociativa por conjuntos de 4 vías, con algoritmo de reemplazo FIFO y políticas de ubicar en escritura (*write-allocate*) y post-escritura (*write-back*)

El espacio virtual está paginado, con un esquema de traducción en 2 niveles. El tamaño de página es de 4 kiB y cada Tabla de Páginas de primer o segundo nivel ocupa exactamente una página. Cada entrada de una Tabla de Páginas ocupa exactamente 4 Bytes, donde el bit más significativo es el bit de residencia y los bits menos significativos son el número de página física. Los restantes son bits de protección y control. Para acelerar la traducción se utiliza una TLB de 256 entradas.

Como almacenamiento secundario tiene un conjunto de discos SSD de 2 TiB cada uno.

El procesador, la memoria principal y el almacenamiento secundario están conectados a través de un bus multiplexado síncrono de 64 bits a 1 GHz.

Contesta a las siguientes preguntas:

- [2p] 1. ☐ Solicito el examen correspondiente a la parte de procesador. Marcando esta casilla se anulará la calificación obtenida en el primer parcial de ejercicios y se reemplazará por la obtenida en este examen.
- [2p] 2. ☐ Solicito el examen correspondiente a la parte de memoria principal y memoria caché. Marcando esta casilla se anulará la calificación obtenida en el segundo parcial de ejercicios y se reemplazará por la obtenida en este examen.

[0.75p] 3. Un núcleo de un procesador Intel Core i9-12900K trabaja a una frecuencia base de 4.0 GHz, y posee dos unidades de ejecución vectoriales AVX-2 capaces de ejecutar 8 operaciones en punto flotante por ciclo cada una. Contesta a las siguientes preguntas:

- (a) **0.25p** ¿Cuál es el rendimiento máximo teórico en GFLOPS de este núcleo? Sabiendo que el procesador cuenta con ocho núcleos idénticos, ¿cuál será el rendimiento máximo teórico del procesador completo?

$$1 \text{ núcleo} \Rightarrow 4.0 \text{ GHz} \times 2 \text{ VU/núcleo} \times 8 \text{ FLOPS/VU} = 64 \text{ GFLOPS/núcleo}$$

$$1 \text{ procesador} \Rightarrow 64 \text{ GFLOPS/núcleo} \times 8 \text{ núcleos} = 512 \text{ GFLOPS}$$

- (b) **0.25p** Se ejecuta en el procesador el siguiente código:

```
float r, a[N];
for( int i = 0; i < N; ++i )
    r = r + a[i];
```

Supongamos que el rendimiento de este código no está limitado por la capacidad de las unidades de ejecución en punto flotante, sino por el sistema de memoria. Si el mismo tiene un ancho de banda de 32 GB/s, ¿cuál será el rendimiento máximo en GFLOPS del procesador al ejecutar el código? Recuerda que cada elemento de tipo `float` ocupa 4 bytes.

Con 32 GB/s de ancho de banda se pueden transferir $32 \cdot 10^9 / 4 = 8 \text{ Gfloats}$ por segundo. Por cada elemento de tipo `float` transferido podemos ejecutar un único FLOP (la suma). Por tanto, el rendimiento máximo estará limitado a 8 GFLOPS.

- (c) **0.25p** Supongamos ahora que el código ensamblador que ejecuta el lazo del apartado anterior incluye una carga, una suma en punto flotante, una suma de enteros con operando inmediato y un salto condicional. En una ejecución concreta del código se ha medido un rendimiento de 6.4 GFLOPS. ¿Cuál ha sido el rendimiento expresado en MIPS?

Dado que se han obtenido 6.4 GFLOPS y por cada FLOP se ejecutan 4 instrucciones, deben haberse ejecutado $6.4 \cdot 10^9 \text{ FLOPS} \times 4 \text{ instrucciones/FLOP} = 25.6 \cdot 10^9 \text{ instrucciones/s} = 25600 \text{ MIPS}$.

Nota: Estas instrucciones se corresponden con un código ensamblador como el siguiente:

```
loop:
    lwc1 $f0, 0($a0)    # carga
    add.s $f2, $f2, $f0 # suma en punto flotante
    addi $a0, $a0, 4    # suma de enteros
    bne $a0, $a1, loop  # salto condicional
```

- [2p] 4. La CPU solicita los datos correspondientes a la dirección virtual 0x0121 27B4. En la entrada de la tabla de páginas de primer nivel se encuentra el contenido 0xA274 14A0 y en la entrada de la tabla de páginas de segundo nivel 0x8002 4605.

(a) **0.25p** Determina en qué campos se divide la dirección virtual.

El tamaño de página es 2^{12} Bytes, por lo que se necesitan 12 bits para el desplazamiento. Cada TP de primer o segundo nivel tiene $\frac{2^{12} \text{ Bytes}}{4 \text{ Bytes/entrada}} = 2^{10}$ entradas. Entonces son 10 bits para PV1, 10 bits para PV2 y 12 bits para desplazamiento.

Dirección virtual		
numPV1	numPV2	Δ_p
10 bits	10 bits	12 bits
0x004	0x212	0x7B4

(b) **0.25p** Determina cómo se divide una dirección física para memoria virtual.

El sistema tiene 1 GiB de memoria física (2^{30} bytes). Como el tamaño de página es de 4 kiB (2^{12} bytes), hay en total $2^{30}/2^{12} = 2^{18}$ páginas físicas. La dirección física se divide de la siguiente forma:

Dirección física	
PF	Δ_p
18 bits	12 bits

(c) **0.4p** Calcula la dirección física de las entradas correspondientes de las tablas de páginas de nivel 1 y 2 si la tabla de páginas de primer nivel se encuentra al comienzo de la memoria física.

El registro base de la TP de nivel 1 es 0, como nos indica el enunciado. En esa tabla tenemos que desplazar hasta la entrada 4, correspondiente al número de PV de nivel 1. Cada entrada ocupa 4 Bytes.

- $DirE1 = 0 + 4 \times 4 = 0x10$

Del contenido de esa entrada, que está en el enunciado, obtenemos la página física en la que comienza la tabla de nivel 2 (los 18 bits menos significativos): 0x014A0. El registro base de la tabla es la dirección física en la que comienza esa tabla: 0x014A0000. Partiendo de esa dirección desplazamos 4 Bytes/entrada hasta la entrada 0x212, correspondiente al número de PV de nivel 2.

- $DirE2 = 0x014A0000 + 0x212 \times 4 = 0x014A 0848$

(d) **0.2p** ¿Cuál es la dirección física resultante del proceso de traducción?

La página física son los 18 bits menos significativos de la entrada de la TP2: 0x24605.

A esa página física le concatenamos el desplazamiento 0x7B4.

La dirección física resultante es 0x246057B4.

(e) **0.1p** ¿Qué información se escribirá en la TLB?

Se escribe el par PV-PF de esta traducción realizada: 0x01212 - 0x24605

(f) **0.2p** Determina el tamaño del Espacio Virtual.

Las direcciones virtuales tienen $10+10+12 = 32 \text{ bits}$, por lo que el espacio virtual es de $2^{32} = 4 \text{ GiB}$.

(g) **0.2p** Si la caché L1 es una caché con índices virtuales y etiquetas físicas (VIPT), calcula en qué conjunto de la caché se encontrará el dato referenciado por la dirección virtual del enunciado.

Las direcciones caché en L1 tienen 6 bits de desplazamiento (el tamaño de línea es de 64 Bytes) y 6 bits de índice (512 líneas en conjuntos de 8 vías $\rightarrow$ 64 conjuntos).

Por tanto los bits de Δ_p de la DV, $0x7B4$ o $0111\ 1011\ 0100_2$ los dividimos de la siguiente forma:

Δ_p	
Índice	Despl.línea
011110_2	110100_2

El índice es $0x1E$ y el desplazamiento $0x34$, por lo que el dato se encuentra en el conjunto $0x1E$.

(h) **0.2p** ¿Qué ventajas e inconvenientes tiene una caché virtual con respecto a otros tipos de caché?

Las cachés virtuales se indexan por las direcciones virtuales en lugar de físicas, y por tanto permiten acceder a memoria caché directamente sin necesidad de traducir las direcciones de memoria. Solo es necesario realizar la traducción en caso de fallo caché. Sin embargo, necesitan información adicional de cada proceso en el directorio caché o bien purgar la caché completa en cada cambio de contexto para evitar problemas graves de seguridad.

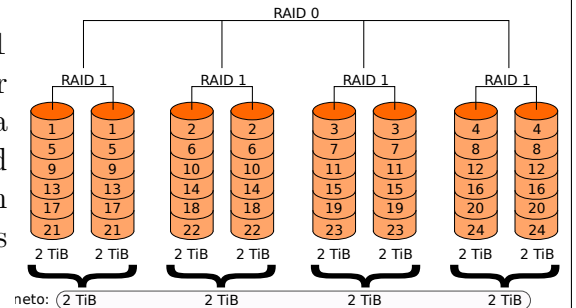
(i) **0.2p** ¿Qué ventaja aporta realizar la traducción en varios niveles con respecto a una traducción directa en un único nivel?

Se reduce considerablemente el espacio necesario para almacenar las tablas de páginas en memoria principal, porque se mantienen en memoria física solo aquellas secciones de la tabla de páginas que corresponden al espacio virtual utilizado.

- [0.5p] 5. El almacenamiento secundario de este sistema está configurado en RAID 1+0 para incrementar la seguridad de la información.

(a) **0.15p** ¿Cuántos discos reales tiene el sistema para la capacidad neta de 8 TiB?

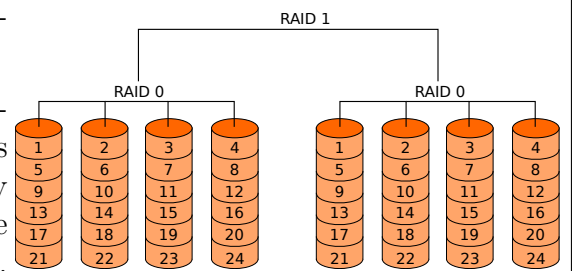
La redundancia en RAID 1+0 la proporciona el RAID 1 (mirroring) sobre el que aplicamos el RAID 0 (striping). Por cada disco de información neta existe un disco espejo con la misma información, por lo que para conseguir la capacidad neta de 8 TiB, que serían 4 discos de 2 TiB, se requieren 8 TiB adicionales con información redundante. Necesitamos en total **8 discos** de 2 TiB.



(b) **0.20p** ¿Qué ventajas o inconvenientes tendría sobre el método actual utilizar RAID 0+1?

RAID 0+1 es un RAID 1 montado sobre un RAID 0. Mantiene la misma cantidad de información redundante que RAID 1+0. Sin embargo, según el controlador RAID que se utilice pueden variar las condiciones que permiten reconstruir el RAID en caso de fallo simultáneo de discos.

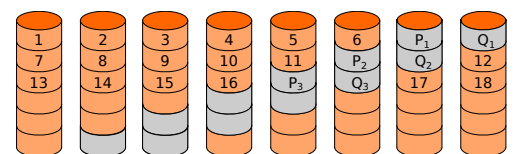
Estos sistemas son capaces de recuperarse ante fallos de discos siempre y cuando no fallen simultáneamente dos discos del mismo RAID 1. En el caso de RAID 1+0, cuando hay un fallo de disco, solo el fallo simultáneo en uno de los siete restantes (en su espejo) produciría un error irreparable. En cambio, en la misma situación en un RAID 0+1, si el fallo simultáneo ocurre en alguno de los cuatro discos que conforman el espejo el sistema fallaría.



(c) **0.15p** Para incrementar la cantidad de información neta en el sistema sin invertir en nuevo hardware, garantizando a la vez que exista cierta redundancia, se implementa un RAID 6. ¿Cuál sería ahora la cantidad de información neta que tendríamos?

Un RAID 6 incorpora doble paridad distribuida, por lo que la información redundante ocupa un tamaño equivalente al de dos unidades de disco.

En este caso, como disponemos de 8 discos de 2 TiB, la **capacidad neta es de 12 TiB** (6 discos) y la información redundante es 4 TiB.



[0.75p] 6. A través del bus indicado, la CPU solicita la transferencia de una página (4 kiB) a memoria física. En cada ciclo de reloj del bus (frecuencia 1 GHz) se pueden transmitir 64 bits de información o una dirección de memoria. El bus permite transacciones de 32 palabras, necesitando para el acceso al primer bloque de 8 palabras en el sistema de almacenamiento 100 ns, y para cada grupo de 8 palabras adicional 30 ns. La transferencia de un bloque a través del bus se puede solapar con el acceso a los siguientes datos. Entre transacciones de bus es necesario esperar 1 ciclo de reloj de bus.

(a) **0.25p** ¿Cuántas transacciones de bus se realizarán para transferir una página completa?

$$1 \text{ transacción} = 32 \text{ palabras} \times 8 \text{ bytes/palabra} = 256 \text{ bytes} = 2^8 \text{ bytes}$$

$$\frac{2^{12} \text{ bytes/página}}{2^8 \text{ bytes/transacción}} = 16 \text{ transacciones/página}$$

(b) **0.50p** Calcula la latencia de la transferencia completa.

El acceso a datos se realiza en bloques de 8 palabras de 64 bits cada una. Como la transacción comprende 32 palabras, es necesario transferir 4 bloques.

Como la frecuencia del bus es 1 GHz, el tiempo de ciclo es $1/10^9 = 1 \text{ ns}$.

Una transacción tiene los siguientes pasos:

- Comienza con el envío de la dirección de memoria (1 ciclo).
- El acceso al primer bloque tarda 100 ns, o 100 ciclos.
- A continuación se transfieren las 8 palabras a razón de 1 palabra/ciclo: 8 ciclos en total.
 - A la vez que se transfieren estas palabras, se accede al siguiente bloque en 30 ns o 30 ciclos.
 - Como el acceso al segundo bloque tarda más que la transferencia del primero, éste será el tiempo que debemos esperar antes de poder transferir el siguiente bloque.
- Esta operación solapada se repite **3 veces**, hasta que se accede al cuarto y último bloque.
- Finalmente se transfiere este último bloque (8 ciclos) y se espera 1 ciclo antes de poder continuar con la siguiente transacción.

$$t_{\text{transacción}} = 1 + 100 + 3 \times \max\{8, 30\} + 8 + 1 = 200 \text{ ns}$$

La latencia de la transferencia de una página es, por tanto:

$$t_{\text{página}} = 16 \text{ transacciones} \times 200 \text{ ns/transacción} = 3200 \text{ ns} = 3.2 \mu\text{s}$$

APELLIDOS: _____ NOMBRE: _____

Hojas adicionales adjuntas: _____

Normas: Escribe los apellidos y el nombre en cada hoja de examen y en cada hoja adicional que utilices. Es posible escribir la respuesta en la misma hoja de cada ejercicio, pero si utilizas hojas adicionales no mezcles en la misma hoja respuestas de ejercicios diferentes: se entregará cada ejercicio por separado. La respuesta a los ejercicios debe estar escrita con bolígrafo. Al terminar, indica en cada hoja de examen las hojas adicionales utilizadas para cada ejercicio.

- [1p] 1. El siguiente código se ejecuta en un procesador MIPS de 5 etapas como el estudiado en clase (IF, ID, EX, MEM, WB). Este procesador cuenta con una unidad de detección de riesgos en la etapa ID y una unidad de anticipación en la etapa EX. El salto se decide en la etapa ID y se utiliza una técnica de predicción de salto.

```
1      addi $t4, $0, 10
2 loop: lw $a1, 0($a0)
3      lw $t0, 0($a1)
4      sw $t0, 0($a0)
5      addi $t4, $t4, -1
6      addi $a0, $a0, 4
7      bne $t4, $0, loop
8      sw $t0, 0($a0)
```

Responde razonadamente a las siguientes cuestiones:

- (a) [0,25p] Identifica las dependencias y su tipo dentro del bucle en el código anterior. ¿Cuáles de estas dependencias provocarán un riesgo en la ejecución?
- (b) [0,25p] Discute la veracidad de la siguiente afirmación: “Si el procesador implementase salto retardado, se puede eliminar la instrucción 4 y, tras la ejecución, el array que recorre \$a0 tendrá los mismos valores que en el resultado original pero desplazados una posición.”.
- (c) [0,25p] Calcula la tasa de acierto del predictor de salto según los siguientes modos de procesamiento de salto que podría implementar el procesador: salto fijo no efectivo, predictor de salto de 1 bit y predictor de salto de 2 bits. En los modos de predicción dinámica, la predicción inicial es de salto no efectivo.
- (d) [0,25p] Para **cada una de las etapas** del pipeline distintas de EX, razona si sería posible **mover** a dicha etapa la unidad de anticipación y en caso afirmativo determina si el rendimiento sería mejor o peor que con la configuración del enunciado.

APELLIDOS: _____ NOMBRE: _____

Hojas adicionales adjuntas: _____

- [1p] 2. Una computadora dispone de un sistema de memoria cache de dos niveles. El primero nivel tiene un tiempo de acierto de 1 ciclo, mientras que el segundo nivel tiene una penalización por fallo de 50 ciclos. El tamaño de línea, tanto a nivel de cache como de memoria principal, es de 16 bytes. El tamaño de palabra es de 4 bytes. Considere el siguiente código:

```
int A[2*N];  
for( i = 0; i < N; ++i ) {  
    r += A[2*i];  
}
```

El tamaño de un `int` es de 4 bytes, y las variables escalares se almacenan en registros del micro. Responda a las siguientes preguntas:

- (a) [0,25p] Indíquese la tasa de fallos local de la cache de nivel 1.
- (b) [0,25p] Indíquese la tasa de fallos global de la cache de nivel 2.
- (c) [0,25p] Sabiendo que el tiempo medio de acceso a memoria durante la ejecución de este programa es de 31 ciclos, indique el tiempo de acierto de la cache de segundo nivel.
- (d) [0,25p] Supóngase un sistema sin caches en el que la matriz A se almacena en una memoria principal con entrelazado de orden inferior a nivel de palabra. Se sabe que en este caso se pueden atender simultáneamente hasta un máximo de cuatro accesos consecutivos del código anterior. Indíquese cuantos módulos componen el sistema.

APELLIDOS: _____ NOMBRE: _____

Hojas adicionales adjuntas: _____

- [1p] 3. Considera un sistema de memoria virtual paginado en 2 niveles. El tamaño del espacio virtual es de 256 TiB y el del espacio físico es de 4 GiB. El tamaño de página es de 64 KiB y se dispone de una TLB totalmente asociativa de 512 entradas. Las tablas de páginas de ambos niveles tienen el mismo número de entradas y la misma estructura. Cada entrada contiene, además del bit de residencia, 7 bits de control.
- (a) [0,3p] Si sabemos que la dirección virtual 0x395B 1880 51A9 se traduce en la dirección física 0xEDDB 51A9 y que la entrada correspondiente en la tabla de segundo nivel comienza en la dirección 0x9ED3 4980, escribe un posible contenido para las entradas de todas las tablas que permitan hacer esta traducción (indicando asimismo sus campos).
 - (b) [0,2p] Escribe los contenidos de las entradas en la TLB implicadas en la anterior traducción. Indica qué cambios se producirían si, justo a continuación, el procesador emite la dirección 0x395B 1880 51E9.
 - (c) [0,2p] Calcula el tamaño conjunto de todas las tablas de páginas de un proceso. Si el tamaño de página se incrementase a 16 MiB manteniendo el tamaño de los espacios virtual y físico y la información de control, ¿aumentaría o disminuiría el espacio ocupado por las tablas? Razona tu respuesta.
 - (d) [0,3p] Si en este ordenador tenemos una caché de 8 MiB asociativa por conjuntos con 16 vías y un tamaño de línea de 64 B, ¿podría aplicarse la técnica VIPT (índice virtual, etiqueta física)? Justifica tu respuesta.

APELLIDOS: _____ NOMBRE: _____

Hojas adicionales adjuntas: _____

[1p] 4. Tenemos un sistema con memoria virtual de páginas de 2 kiB que conecta la memoria principal con el almacenamiento secundario mediante un bus síncrono de 128 bits a 100 MHz. Las transferencias de las páginas están gestionadas por una DMA a través de este bus en bloques del tamaño de la anchura del bus, necesitando 280 ns para la lectura del primer bloque y 100 ns para el resto en una operación de lectura de una página. Tanto el envío de 128 bits a través del bus como el de la dirección al controlador del almacenamiento secundario requieren un ciclo. Las transferencias de datos leídos más recientemente pueden solaparse con la lectura de los siguientes. Sabiendo que la DMA realiza la lectura de una página en una única transacción, calcula:

- (a) [0,4p] Latencia para la lectura de una página.
- (b) [0,2p] Ancho de banda del bus para las operaciones de lectura de página desde el almacenamiento secundario.

El almacenamiento secundario tiene una capacidad neta de 8 TiB, repartidos en cuatro discos que permiten accesos concurrentes. Queremos tener un RAID con información redundante y solicitamos precios de discos a varios proveedores que nos proporcionan la siguiente información:

- Compañía MACWEL: discos de 2 TB, serie 3NV-0001 450€, serie 4NW-0001 490€.
- Compañía TEABATE: disco de 2 TB, serie WW-0001-YN1 440€.
- Compañía EastDigi: discos de 2 TB, serie ED-A-0001-W 430€, serie ED-A-0015-Y 460€, serie ED-B-1020-C 500€.

Teniendo en cuenta que las características técnicas de todos estos discos son similares y sabiendo que la inversión no puede superar los 2500€, contesta:

- (c) [0,2p] ¿Qué tipo de RAID implementarías con el mejor rendimiento? ¿Por qué?
- (d) [0,2p] ¿Cuál sería la combinación de discos que utilizarías y cuál sería su coste? Razona la elección.

1. (a) Dentro del bucle (líneas 2 a 7) hay las siguientes dependencias:

- $2 \rightarrow 3$: Dependencia verdadera (\$a1). Provoca riesgo RAW al no poder anticipar \$a1 hasta finalizar la etapa MEM.
- $3 \rightarrow 4$: Dependencia verdadera (\$t0). Provoca riesgo RAW al no poder anticipar \$t0 hasta finalizar la etapa MEM.
- $6 \rightarrow 2, 4$: Antidependencia (\$a0).
- $5 \rightarrow 7$: Dependencia verdadera (\$t4). Provoca riesgo RAW al tener que esperar a que \$t4 se escriba en el banco de registros.

En memoria hay una antidependencia entre 4 y 2 en la dirección [\$a0 + 0] que no se ha calificado negativamente.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14
addi \$t4, \$0, 2	IF	ID	EX	ME	WB									
loop: lw \$a1, 0(\$a0)		IF	ID	EX	ME	WB								
lw \$t0, 0(\$a1)			IF	ID	—	EX	ME	WB	← RAW					
sw \$t0, 0(\$a0)				IF	—	ID	—	EX	ME	WB	← RAW			
addi \$t4, \$t4, -1					IF	—	ID	EX	ME	WB				
addi \$a0, \$a0, 4						IF	ID	EX	ME	WB				
bne \$t4, \$0, loop							IF	ID	—	EX	ME	WB	← RAW	

- (b) Es falso. El resultado del código sería totalmente diferente, porque la instrucción 8 del hueco de retardo sobrescribirá en cada iteración el valor que leerá la instrucción 2 en la siguiente iteración. Si en la instrucción 3 en \$t0 no se carga una dirección de memoria, el resultado será un error de acceso a memoria en la segunda iteración.
- (c) El salto se ejecuta 10 veces. Con salto fijo no efectivo sólo se acierta en la última ejecución: $TA = 1/10$. Con un predictor de 1 bit se fallará en la primera y la última ejecución del salto: $TA = 8/10$. Con un predictor de 2 bits se fallará en las 2 primeras y en la última ejecución: $TA = 7/10$.
- (d) IF En la etapa IF no tiene sentido colocar la unidad de anticipación porque todavía no se ha decodificado la instrucción y desconocemos los registros que utilizará.
 ID En la etapa ID, anticipando los valores a la salida del banco de registros, favorecería a la instrucción de salto (la única para la que esta opción es útil) y se elimina el riesgo entre 5 y 7. Sin embargo, en las dependencias con las instrucciones de carga de memoria (lw) es necesario esperar un ciclo adicional. El rendimiento por tanto es peor que en el código original.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
addi \$t4, \$0, 2	IF	ID	EX	ME	WB										
loop: lw \$a1, 0(\$a0)		IF	ID	EX	ME	WB									
lw \$t0, 0(\$a1)			IF	ID	—	—	EX	ME	WB						
sw \$t0, 0(\$a0)				IF	—	—	ID	—	—	EX	ME	WB			
addi \$t4, \$t4, -1							IF	—	—	ID	EX	ME	WB		
addi \$a0, \$a0, 4										IF	ID	EX	ME	WB	
bne \$t4, \$0, loop											IF	ID	EX	ME	WB

MEM En esta etapa, anticipando el valor del registro a la entrada del banco de memoria, se elimina el riesgo entre 3 y 4 porque la instrucción sw podrá obtener el valor de \$t0 del registro de segmentación MEM/WB (la instrucción 4 necesita el valor de \$a0 en su etapa ID para calcular la dirección destino y \$t0 en su etapa MEM para guardarlo en memoria). El riesgo entre 2 y 3 se incrementa en un ciclo al no poder anticipar el valor de \$a1 a la ALU, que necesita la segunda lw para calcular la dirección de memoria a la que acceder. El rendimiento por tanto no varía.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14
addi \$t4, \$0, 2	IF	ID	EX	ME	WB									
loop: lw \$a1, 0(\$a0)		IF	ID	EX	ME	WB								
lw \$t0, 0(\$a1)			IF	ID	—	—	EX	ME	WB					
sw \$t0, 0(\$a0)				IF	—	—	ID	EX	ME	WB				
addi \$t4, \$t4, -1							IF	ID	EX	ME	WB			
addi \$a0, \$a0, 4								IF	ID	EX	ME	WB		
bne \$t4, \$0, loop									IF	ID	—	EX	ME	WB

WB En la etapa WB no tiene sentido colocar la unidad de anticipación pues (entre otras cosas) no existen registros de segmentación posteriores desde los cuales se puedan anticipar datos.

2. El acceso al array es secuencial saltando un elemento de tipo entero en cada acceso (de cada 8 Bytes en el array se leen 4).
- Cada fallo en la cache L1 trae de la cache L2 16 bytes, que contienen datos útiles para esa iteración y la siguiente. La tasa local de fallos es por tanto 50%
 - Cuando ocurre un fallo en la cache L1 también ocurre en la cache L2, porque no se reutilizan datos en L2. Por tanto, la tasa local de fallos en L2 es del 100%. La tasa global de fallos es el número de fallos en L2 dividido por el número de accesos a L1. Entonces, esa tasa global es del 50% igual que en L1.
 - Para calcular el TMA del sistema caché necesitamos las tasas locales de fallos en L1 y L2, que son del 50% y 100% respectivamente. El tiempo de acceso a L1 y la penalización por fallo de L2 nos las proporciona el enunciado. Entonces tenemos que $1 + 0.5(t_{accesoL2} + 1 \times 50) = 31 \rightarrow t_{accesoL2} = 10$
 - En cada acceso a datos se cargan 4 palabras desde memoria principal a memoria cache. Esas 4 palabras son suficientes para completar 2 iteraciones del código. Como el entrelazamiento es de orden inferior a nivel de palabra, cada una de estas palabras se encontrará en un módulo diferente. Para que puedan accederse exactamente a los datos de 4 iteraciones de forma simultánea son necesarios por tanto el doble de módulos: necesitamos 8 módulos de memoria
3. (a) $|V| = 256 \text{ TiB} = 2^{48} B \Rightarrow \text{DV de 48 bits}$
 $|F| = 4 \text{ GiB} = 2^{32} B \Rightarrow \text{DF de 32 bits}$
 $|P| = 64 \text{ KiB} = 2^{16} B \Rightarrow \Delta \text{ de 16 bits}$

Si las tablas de páginas de ambos niveles tienen el mismo número de entradas, los dos campos para indexarlas tienen el mismo tamaño.

Índ TP_1	Índ TP_2	Δ	núm PF	Δ
395B	1880	51A9	EDDB	51A9
16 bits	16 bits	16 bits	16 bits	16 bits

La tabla de primer nivel nos permite localizar la tabla de segundo nivel correspondiente. Por tanto, en el número de página física almacenado tendremos la parte correspondiente a la dirección de entrada de la TP_2 que leemos.

	R	control	núm PF
$TP_1[395B]$	1	—	9ED3
	1 bit	7 bits	16 bits

Suponiendo 0s en los bits de control tendríamos almacenado el contenido 0x809ED3. La tabla de segundo nivel ya nos da el número de página física resultado de la traducción. Así:

	R	control	núm PF
$TP_{2,395B}[1880]$	1	—	EDDB
	1 bit	7 bits	16 bits

Análogamente, suponiendo 0s en los bits de control, tendríamos almacenado el contenido 0x80EDDB.

- (b) La TLB contiene pares página virtual-página física de las últimas traducciones.

núm PV	núm PF
395B 1880	EDDB
32 bits	16 bits

La dirección 0x395B 1880 51E9 se encuentra en la misma página: sólo cambian los bits dentro del desplazamiento Δ . Por tanto, la TLB no cambia.

- (c) $|TP_1| = 2^{16} \text{ entradas} \times (1 + 7 + 16) \text{ bits} = 2^{16} \times 3 \text{ B} = 192 \text{ KiB}$
Cada una de las TP_2 tiene el mismo tamaño, y tenemos una tabla para cada entrada de la TP_1
 $|TP| = |TP_1| + 2^{16} \times |TP_{2,i}| = |TP_1| \times (1 + 2^{16}) \approx 12 \text{ GiB}$

Si incrementamos el tamaño de página estamos reduciendo el número de páginas del sistema, por lo que las tablas tendrán menos entradas (por haber menos páginas virtuales) y estas entradas serán de menor tamaño (ya que necesitaremos menos bits para guardar el número de página física puesto que también tendremos menos páginas físicas).

- (d) $|C| = |ZA| = 8 \text{ MiB} = 2^{23} B$
 $|l| = 64 B = 2^6 B \Rightarrow \Delta_l \text{ de 6 bits}$
 $\# \text{líneas} = |ZA|/|l| = 2^{23-6} = 2^{17}$
 $\# \text{conjuntos} = \# \text{líneas} / 16 \text{ vías} = 2^{17-4} = 2^{13} \Rightarrow \text{índice de 13 bits}$

núm PF	Δ	etiqueta	índice	Δ_l
16 bits	16 bits	13 bits	13 bits	6 bits

Para poder aplicar la técnica VIPT, el índice que utilizamos para acceder a la caché tiene que coincidir dentro de los bits de desplazamiento de página Δ (la parte de la DV que no se traduce). En este caso, $16 \leq (13 + 6)$ hace que no sea posible.

4. (a) *El bus de 128 bits indica que se leen 16 bytes de cada vez. Así, el número de lecturas para transferir una página sería:*

$$NL = \frac{2 \text{ kiB}}{2^4 \text{ B}} = \frac{2^{11}}{2^4} = 2^7 = 128$$

La frecuencia del bus es de 100 MHz, con lo cual el período es:

$$T = \frac{1}{100 \times 10^6} = 10^{-8} \text{ s} = 10 \text{ ns}$$

La transferencia de una página requiere:

- 1 ciclo para enviar la dirección.
- 280 ns para leer los primeros 16 bytes. Corresponden a 28 ciclos.
- 100 ns para leer cada uno de los 127 grupos de 16 bytes restantes. Serían $10 \times 127 = 1270$ ciclos.
- 1 ciclo para transferir los últimos 16 bytes.

En total una página necesita $1 + 28 + 1270 + 1 = 1300$ ciclos, siendo la latencia de transferencia de una página:

$$L = 1300 \times 10 = 13000 \text{ ns} = 13 \mu\text{s}$$

- (b) *El ancho de banda sería:*

$$AB = \frac{2 \text{ kiB}}{13 \times 10^{-6} \text{ s}} \approx 0,154 \times 10^9 \frac{\text{B}}{\text{s}} = 154 \frac{\text{MB}}{\text{s}}$$

- (c) *Para accesos concurrentes se emplean los RAID 4, 5 ou 6 usando como mínimo 5 discos. Para 6 discos, con estos precios necesitaríamos un mínimo de 2580€, superior al presupuesto disponible. Por tanto, habría que escoger entre RAID 4 y RAID 5, dando mejor rendimiento el RAID 5.*
- (d) *Para un RAID 5 tenemos que comprar 5 discos, y para mayor fiabilidad es preferible que sean de distinta serie, así que segun los precios una posible elección sería escoger un disco de cada tipo excepto el ED-B-1020-C de EastDigi que es el más caro de esta casa, con un gasto final de 2270€.*

ESTRUCTURA DE COMPUTADORES

27 de enero de 2021

APELLIDOS: _____ NOMBRE: _____

1. [1p] Un procesador implementa un camino de datos segmentado con 10 etapas. Cada una de las etapas se completa en un ciclo de reloj. En este procesador todas las instrucciones pasan por todas las etapas, y las instrucciones de salto condicional (bne, beq) son las únicas que pueden provocar un riesgo. Cuando una instrucción de salto condicional termina de ejecutar la etapa 1, el *pipeline* se bloquea hasta que se evalúa la condición de salto.

Sobre este procesador ejecutamos un código que tarda 1809 ciclos tras la ejecución efectiva de 1000 instrucciones, de las cuales 200 son instrucciones de salto condicional y 400 son operaciones en punto flotante.

- a) [0.2p] Determina en qué etapa del camino de datos se evalúa la condición de salto.
- b) [0.4p] Calcula cuál de las siguientes acciones aplicadas individualmente mejora el rendimiento del procesador en mayor medida:
 - Reducir la longitud del *pipeline* en 2 etapas, sin que afecte a los riesgos que se pudiesen producir.
 - Mover la evaluación de la condición de salto a la etapa 4
 - Utilizar una estrategia de predicción de salto que en el código descrito tiene una tasa de acierto de un 20 %
 - Utilizar una estrategia de salto retardado, sabiendo que en término medio disponemos de 1,5 instrucciones para rellenar el hueco de retardo (el resto se rellena con instrucciones *nop*).
- c) [0.4p] Al compilar el código fuente que produjo el código máquina anterior con nuestro nuevo compilador, al que hemos denominado *ecc*, obtenemos un código de 1400 instrucciones efectivas de las que 600 son operaciones en punto flotante, que tarda 2200 ciclos en ejecutarse. Buscamos una métrica de rendimiento con la que mostrar que *ecc* supera al compilador anterior.
 - Determina con qué métrica podríamos alardear más de nuestro nuevo compilador: tiempo de ejecución, MIPS o GFLOPS.
 - Desde el punto de vista de un potencial usuario de *ecc*, ¿sería una comparación justa?

2. [1p] Los códigos de tipo “streaming”, entre los que se encuentran los reproductores de vídeo o audio, se caracterizan por consumir grandes cantidades de datos, pero no reusarlos. Considérese el siguiente código:

```
int A[N];
for( i = 0; i < N; ++i ) {
    r += A[i];
}
```

Contesta a las siguientes preguntas:

- Asumiendo una cache de correspondencia directa de 64 kB con tamaño de línea de 32 bytes, y que el tamaño de un `int` es de 4 bytes, ¿cuál será la tasa de fallos de este código? ¿Qué tipo de fallos es el más común en este código?
- ¿Cómo varía la tasa de fallos al variar el tamaño del array? ¿Y al variar el tamaño de la cache?
- ¿Cómo varía la tasa de fallos para tamaños de línea de 16, 64 y 128 bytes?
- Supongamos otro código que presenta las siguientes tasas de fallo según el tamaño de bloque:

8: 4 %	16: 3 %	32: 2 %	64: 1.5 %	128: 1 %
--------	---------	---------	-----------	----------

¿Cuál es el tamaño de línea L óptimo para una latencia de acceso de $20 \cdot L$ ciclos?

3. [1p] Considera un computador con un sistema de memoria virtual paginado, una TLB, una única caché y que soporta varios procesos. La memoria es direccionable a nivel de byte y el tamaño del espacio virtual es de 256 TB. La TLB tiene 128 entradas y es totalmente asociativa. La caché, de 4 MB, es asociativa por conjuntos de 8 vías y el tamaño de línea es de 64 B.

- [0,3p] Calcula el tamaño de página mínimo que permitiría aplicar la técnica VIPT (índice virtual, etiqueta física) en este sistema.
- [0,1p] ¿Cuántas páginas virtuales se soportarían por proceso?
- [0,6p] Justifica razonadamente si las siguientes afirmaciones son verdaderas o falsas:
 - Aumentar el tamaño de página haría que se redujese el tamaño de la tabla de páginas.
 - Aumentar el tamaño de página haría que se redujese la fragmentación interna.
 - Duplicar el tamaño de la TLB impediría aplicar la técnica VIPT.
 - Sería posible seguir utilizando la técnica VIPT en un sistema de memoria virtual segmentada en el que la longitud máxima de segmento fuese el tamaño de página calculado anteriormente.
 - Sería eficiente aplicar la técnica de escritura directa (write-through) en el sistema de memoria virtual.
 - Nunca sería útil tener un espacio de memoria física mayor que el espacio de memoria virtual.

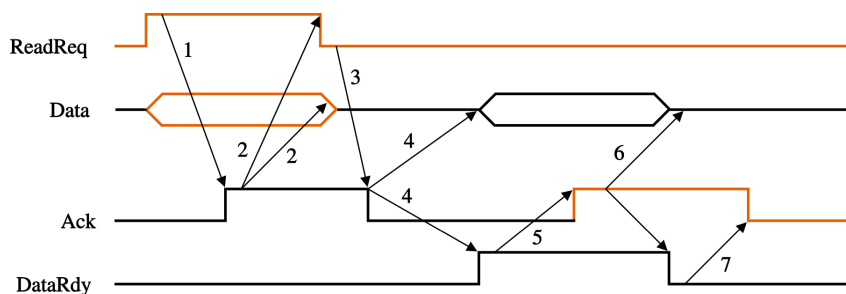
4. [1p] Sea un computador con las siguientes características:

- Las direcciones de memoria y las palabras son de 64 bits.
- Tiene una jerarquía caché de dos niveles. El primer nivel tiene una tasa de fallo del 10 % y un tiempo de acierto de 5 ns. El segundo nivel tiene un tiempo de acierto de 8 ns y una tasa de fallo del 40 %. El tiempo medio de acceso a todo el sistema de memoria es de 9 ns cuando ambos niveles tienen un tamaño de línea de 8 palabras.
- La caché de segundo nivel se conecta mediante un bus síncrono de 64 bits a la memoria principal, en el que tanto una transferencia de 64 bits como el envío de la dirección de memoria requieren 1 ciclo de reloj.
- El sistema de memoria y el bus soportan accesos a bloques de 8 palabras de 64 bits. Se necesitan dos ciclos de reloj entre dos operaciones de bus (se supondrá el bus libre antes de cada acceso). Esto es, entre el envío de dos bloques de palabras, el bus esperará 2 ciclos. La memoria tarda lo equivalente a 20 ciclos del bus en acceder a las primeras 4 palabras del bloque; y luego cada grupo adicional de cuatro palabras del mismo bloque se lee en 13 ciclos de bus. Se considerará que la transferencia de los datos por el bus puede solaparse con la lectura de las cuatro palabras siguientes.

Calcular:

- a) [0.5p] La frecuencia del bus síncrono que comunica la caché de segundo nivel con la memoria principal. El ancho de banda y la latencia para la lectura, desde la caché de segundo nivel, de 128 palabras de memoria, en el caso de transferencias de bloques de 8 palabras.
- b) [0.3p] Supongamos que este bus soporta transacciones de bloques de 4 palabras y que la penalización por fallo de la caché de segundo nivel es de 54 nanosegundos cuando trabaja con líneas de 4 palabras. ¿Tendría sentido reemplazar este bus síncrono de memoria por un bus asíncrono donde cada paso del protocolo *handshaking* requiere de 30 nanosegundos en transferencias de bloques de 4 palabras? ¿Por qué? Justifica numéricamente tu respuesta. ¿En el caso general tiene sentido reemplazar un bus de memoria síncrono por uno asíncrono?
- c) [0.2p] Este computador tiene un total de 1024 GB de información muy valiosa almacenada en disco. Dispongo de hasta 1000 euros para montar un RAID y sabiendo que cada disco de 256 GB me cuesta 100 euros, ¿qué configuración RAID escogería?

Puedes ayudarte de la siguiente figura para el apartado de bus asíncrono:



SOLUCIÓN

1. a) Si no hubiese ningún riesgo, el código se ejecutaría en 1009 ciclos (10 ciclos de la primera instrucción y un ciclo adicional por cada una de las 999 instrucciones restantes). Por lo tanto los $1809 - 1009 = 800$ ciclos restantes están provocados por las 200 instrucciones de salto, a razón de 4 ciclos cada una, lo que quiere decir que la condición de salto se evalúa en la etapa $1 + 4 = 5$. Lo podemos ver fácilmente con un diagrama:

salto	E1	E2	E3	E4	E5	E6	E7	...
siguiente		-	-	-	-	E1	E2	...

Otras respuestas correctamente razonadas también fueron consideradas como válidas.

- b)
- Reducir el *pipeline* en 2 etapas sólo nos aporta una mejora de 2 ciclos, por lo que descartamos esta opción.
 - Si el salto se decide en la etapa 4, cada instrucción de salto provocará un bloqueo de 3 ciclos en lugar de 4. La aceleración es $\frac{1809}{1009+200*3} = \frac{1809}{1609} = 1,12$
 - Si aplicamos el predictor de salto y acierta el 20 %, significa que de las 200 instrucciones de salto habrá 40 aciertos en los que no se producirá un riesgo, y 160 en las que el *pipeline* se detendrá durante 4 ciclos. El número de ciclos tras aplicar esta mejora es $1009+160*4 = 1649$, y la aceleración es $1809/1649 = 1,097$
 - Utilizando salto retardado se reduce el riesgo provocado por cada salto en 1,5 ciclos, por lo que ahora el riesgo en cada salto será de $4 - 1,5 = 2,5$ ciclos. La aceleración es $\frac{1809}{1009+200*2,5} = \frac{1809}{1509} = 1,20$. Ésta es la opción más beneficiosa.
- c)
- El tiempo de ejecución (que sería lo más justo) no nos sirve porque con el nuevo compilador el código tarda más ciclos en ejecutarse. La máquina en la que ejecutamos los códigos es la misma, porque estamos comparando los compiladores, y por lo tanto el tiempo de ciclo (T_c) es constante y el tiempo de ejecución es proporcional al número de ciclos.
 - Con la métrica MIPS, aunque no podemos calcular el valor de MIPS exacto de cada código porque desconocemos el tiempo de ciclo, comparando ambos códigos podemos calcular que éste es $\frac{1400/((2200 \times \cancel{T_c} \times 10^6))}{1000/((1809 \times \cancel{T_c} \times 10^6))} = \frac{1400/2200}{1000/1809} = 1,15$ veces mayor.
 - Lo mismo sucede con la métrica GFLOPS. No podemos calcular su valor exacto, pero el valor del nuevo compilador es $\frac{600/((2200 \times \cancel{T_c} \times 10^9))}{400/((1809 \times \cancel{T_c} \times 10^9))} = \frac{600/2200}{400/1809} = 1,23$ veces mayor.
 - Por lo tanto la mejor opción es usar la métrica GFLOPS, aunque no sería una comparación justa ya que el compilador original genera un código más eficiente y lo que buscamos en un compilador es que produzca un código que se ejecute lo más rápido posible.
2. a) Dado que el tamaño de línea es de 32 bytes y cada entero ocupa 4 bytes, cada lectura de una línea de memoria traerá 8 enteros a cache. Asumiendo que **r** e **i** están almacenadas en registros del micro y que la dirección de inicio de **A** está alineada a una palabra de memoria, y teniendo en cuenta que no hay reuso y que el recorrido del array es secuencial, tendremos un fallo forzoso por cada 8 accesos, resultando una tasa de fallos de $\frac{1}{8}$. Todos los fallos serían forzosos, no es posible que existan fallos de capacidad ni de conflicto al no haber reuso.

- b) Ni el tamaño del array ni el de la cache influyen. Lo único que afecta a la tasa de fallos de este código es el tamaño de la línea cache y el tamaño de cada elemento del array.
- c) Para 16, 64 y 128 bytes podríamos almacenar, respectivamente, 4, 16, y 32 enteros en cada línea cache. Las tasas de fallos serían, por tanto, de $\frac{1}{4}$, $\frac{1}{16}$, y $\frac{1}{32}$, respectivamente.
- d) Asumiendo que el tamaño de línea no influye en el tiempo de acierto, la parte del tiempo medio de acceso a memoria correspondiente a la penalización por fallo en cada uno de los casos será:
- Líneas de 8B: $8 \cdot 20 \cdot 0,04 = 6,4 \text{ ciclos}$
 - Líneas de 16B: $16 \cdot 20 \cdot 0,03 = 9,6 \text{ ciclos}$
 - Líneas de 32B: $32 \cdot 20 \cdot 0,02 = 12,8 \text{ ciclos}$
 - Líneas de 64B: $64 \cdot 20 \cdot 0,015 = 19,2 \text{ ciclos}$
 - Líneas de 128B: $128 \cdot 20 \cdot 0,01 = 25,6 \text{ ciclos}$

Por tanto, en este caso el mejor tamaño de línea es el de 8B, dado que su incremento no mejora la tasa de fallos lo suficiente como para compensar el incremento en la penalización de fallo.

3. a) $|V| = 256 \text{ TB} = 2^{48} B \Rightarrow \text{DV de 48 bits}$

$$|C| = |ZA| = 4 \text{ MB} = 2^{22} B$$

$$|l| = 64 \text{ B} = 2^6 B \Rightarrow \Delta_l \text{ de 6 bits}$$

$$\# \text{líneas} = |ZA|/|l| = 2^{22-6} = 2^{16}$$

$$\# \text{conjuntos} = \# \text{líneas} / 8 \text{ vías} = 2^{16-3} = 2^{13} \Rightarrow \text{índice de 13 bits}$$

Núm. PV	Δ_p	Núm. PF	Δ_p	Etiqueta	Índice	Δ_l
48-p bits	p bits	??-p bits	p bits	?? bits	13 bits	6 bits

Para poder aplicar la técnica VIPT, el índice que utilizamos para acceder a la caché tiene que coincidir dentro de los bits del desplazamiento de página (la parte de la DV que no se traduce). Por tanto, $p \geq (13 + 6) \Rightarrow |P| \geq 2^{19} B$.

- b) Cada proceso tiene su propio espacio de direcciones virtuales. Nos quedan hasta $48 - 19 = 29$ bits para expresar el número de página virtual. Por tanto, $\#PV \leq 2^{29}$.
- c)
- VERDADERO. Al aumentar el tamaño de página estamos reduciendo el número de páginas del sistema, por lo que la tabla tendrá menos entradas (por tener menos páginas virtuales) y estas entradas serán de menor tamaño (ya que necesitaremos menos bits para guardar el número de página física al que traducimos puesto que también tendremos menos páginas físicas).
 - FALSO. Al aumentar el tamaño de página estamos aumentando la probabilidad de tener espacio desaprovechado en cada una de ellas.
 - FALSO. La TLB es una caché totalmente asociativa que guarda las últimas traducciones. Así, contiene pares {página virtual, página física} (junto con información adicional de control). Por tanto, duplicar su tamaño simplemente nos permitirá tener más pares. No tiene influencia en la aplicación de la técnica VIPT, como pudimos ver en el apartado (a).
 - FALSO. Si bien es cierto que el desplazamiento dentro del segmento no se traduce, en el proceso traducción éste se suma a la dirección física donde comience

el segmento en cuestión. Por tanto, dichos bits de la dirección virtual no podemos utilizarlos para indexar la caché ya que, típicamente, no coincidirán con los últimos bits de la dirección física a la que se traduce.

- FALSO. En escritura directa llevaríamos cada escritura en memoria principal al nivel inferior. El nivel inferior en la jerarquía es el disco duro. Sea de estado sólido o magnético con acceso rotacional, su tiempo de acceso es varios órdenes de magnitud mayor.
- FALSO. En un sistema que soporta varios procesos, como el que estamos contemplando, nos permitiría tener en memoria física más páginas de los distintos procesos que hiciesen los cambios de contexto más rápidos. Otro ejemplo sería el uso de varias máquinas virtuales en un equipo.

4. a) Si tiempo medio acceso son $9 \text{ ns} = 5 + 0,1 \times (8 + 0,4 \times PF)$, por tanto $PF = 80 \text{ ns}$. Tenemos 1 ciclo enviar *dir* + 20 ciclos para acceder pal 0-3 + max(4 ciclos de envío pal 0-3, 13 ciclos acceso pal 4-7) + 4 ciclos envío pal 4-7 + 2 ciclos espera = 40 ciclos. Si 40 ciclos tardan 80 ns (PF) entonces $T_{\text{ciclo}} = 2 \text{ ns}$ y por tanto la frecuencia es su inversa: $f = 500 \text{ MHz}$.

Si 1 transacción son 40 ciclos y hay 16 transacciones (128 palabras / 8 líneas) la latencia es de $16 \times 40 \text{ ciclos} = 640 \text{ ciclos}$, que son 1280 ns. El ancho de banda se calcula como $\frac{128 \times 8 \text{ B}}{1280 \times 10^{-9} \text{ s}} = 800 \text{ MB/s}$.

- b) Para resolver este apartado no se necesitan cálculos, más allá de saber que al necesitar 7 pasos el protocolo de handshaking, una transacción por el bus asíncrono tardará un mínimo de $7 \times 30 = 210 \text{ ns}$, independientemente de lo que tarde una transferencia de datos por el bus o el acceso a una dirección de memoria. Por lo tanto, el bus síncrono, que tarda 54 ns será más rápido. Se espera una justificación sobre el uso de buses síncronos para el intercambio de datos entre dispositivos rápidos.
- c) Como el enunciado me dice que los datos son muy valiosos, la configuración más adecuada es un RAID 1 (preferiblemente 1+0) aunque sacrifique rendimiento.

APELLIDOS: _____ NOMBRE: _____

Bloque A

[0.75p] 1. Un programa consta de 10^9 instrucciones. El 60% son operaciones en punto flotante que tardan 10 ciclos en ejecutarse, mientras que las restantes tardan 5 ciclos. El programa se ejecuta en dos procesadores, S1 y S2, que implementan el mismo conjunto de instrucciones. S1 es un procesador multiciclo a 2 GHz y S2 un procesador segmentado a 500 MHz. En S2 el programa tarda 3 segundos en ejecutarse.

(a) Calcula el CPI de S1 y S2 para este programa.

$$CPI_{S1} = \frac{(0.6 \times 10 + 0.4 \times 5) \times 10^9 \text{ ciclos}}{1 \times 10^9 \text{ instr}} = 8 \text{ ciclos/instr}$$

El procesador segmentado puede solapar la ejecución de instrucciones, por lo que su CPI se reducirá. Al conocer el tiempo de ejecución se puede calcular el número de ciclos y por tanto el CPI.

$$CPI_{S2} = \frac{3 \text{ s} \times 0.5 \times 10^9 \text{ ciclos/s}}{10^9 \text{ instr}} = 1.5 \text{ ciclos/instr}$$

(b) Calcula el rendimiento de S1 según la métrica FLOPS.

$$T_{CPU} = \frac{(0.6 \times 10 + 0.4 \times 5) \times 10^9 \text{ ciclos}}{2 \times 10^9 \text{ ciclos/s}} = 4 \text{ s}$$

$$\frac{0.6 \times 10^9 \text{ flops}}{4 \text{ s}} = 0.15 \times 10^9 \text{ FLOPS} = 150 \text{ MFLOPS}$$

(c) Usando la Ley de Amdahl, determina en qué proporción debería mejorarse la unidad de punto flotante de S1 para que la ejecución de este programa tarde lo mismo en ambos procesadores.

$$\left. \begin{array}{l} T_{S1} = 4 \text{ s} \\ T_{S2} = 3 \text{ s} \end{array} \right\} A = \frac{4}{3}$$

$$f_m = \frac{0.6 \times 10}{0.6 \times 10 + 0.4 \times 5} = \frac{3}{4}$$

$$\frac{4}{3} = \frac{1}{1 - 3/4 + \frac{3/4}{A_m}}$$

$$A_m = \frac{3}{2} = 1.5$$

[2p] 2. Un computador utiliza memoria virtual paginada en dos niveles y cuenta con 32 KiB (2^{15} bytes) de memoria principal y 1 MiB (2^{20} bytes) de memoria virtual. La memoria se divide en 128 páginas físicas (2^7). Cada tabla de páginas involucrada en el proceso de traducción ocupa exactamente una página física, y cada entrada comienza con 1 bit de residencia, seguido de 1 bit de modificación, 23 bits adicionales de protección y control, y el número de página física.

- (a) Indica en qué campos y con qué valores (en hexadecimal) se dividiría la dirección virtual 0x5D392 y la dirección física 0x72B4.

Si los 2^{15} bytes de memoria se dividen en 2^7 páginas, el tamaño de página es 2^8 bytes. Cada entrada de una TP ocupa $1+1+23+7 = 32$ bits = 4 bytes. Cada tabla de páginas tiene $256/4 = 64$ entradas. Una DV (20 bits) se divide en 6 bits de PV1, 6 bits de PV2 y 8 bits de Δ_p . Una DF (15 bits) se divide en 7 bits de PF y 8 bits de Δ_p . Entonces las direcciones se dividen de la siguiente forma:

PV1	PV2	Δ_p	PF	Δ_p
01 0111	01 0011	1001 0010	0111 0010	1011 0100
0x17	0x13	0x92	0x72	0xB4
6 bits	6 bits	8 bits	7 bits	8 bits

- (b) Calcula cuánto ocuparían todas las tablas de nivel 2 de un proceso

Una DF (15 bits) se divide en 7 bits de PF y 8 bits de Δ_p . Cada entrada de una tabla contiene $1 + 1 + 23 + 7 = 32$ bits = 4 bytes. Cada tabla tiene $256/4 = 64$ entradas. Hay por tanto una tabla de nivel 1 y 64 tablas de nivel 2. Como cada una ocupa 256 bytes, ocupan en total $64 * 256 = 16$ KiB.

- (c) En la traducción de las siguientes direcciones virtuales (DV), se indica en la siguiente tabla el contenido de las tablas de páginas de nivel 1 (PT1) y 2 (PT2), en hexadecimal. Si es posible realizar la traducción, indica la dirección física correspondiente (DF). En caso contrario, justifica por qué no es posible hacerla.

DV	TP1	TP2	DF
905D7	8341 1C00	6529 AC32	no es posible
72A42	9B21 6B30	C2A8 04A5	0x2542
C3B12	D281 6A03	8A21 BC52	0x5212

En el caso indicado no es posible hacer la traducción porque el Bit de Residencia de la entrada en TP2 es 0. Para el resto, los 7 últimos bits de la entrada en TP2 indican la PF y concatenamos los 8 últimos bits de la DV.

- (d) Para la primera dirección del apartado anterior (0x905D7), determina la dirección de la entrada en la tabla de páginas de nivel 2.

La entrada en TP1 indica que la TP2 se encuentra en la página física 0. La DV indica que debemos acceder a la entrada 5 de esa tabla. Como cada entrada ocupa 4 bytes, la dirección es $0 + 5 \times 4 = 0x0014$.

- (e) Supongamos que este sistema tuviese 256 KiB de memoria virtual y un proceso utiliza todo el espacio virtual. Las direcciones virtuales se dividirían en 10 bits de página virtual y 8 bits de desplazamiento. Determina la partición más eficiente de esos 10 bits de PV en dos niveles de paginación.

Tendremos una TP de nivel 1 y tantas TP de nivel 2 como entradas tenga la TP de nivel 1. Teniendo en cuenta que las tablas ocuparán siempre una página (2^8 bytes), para minimizar el espacio total lo ideal es que haya el menor número de TP de nivel 2 y estén completas (2^6 entradas) para que no exista fragmentación interna. Por tanto, la mejor opción es 4 bits de PV1 y 6 de PV2.

APELLIDOS: _____ NOMBRE: _____

Bloque A

- [0.5p] 3. Tenemos discos duros de capacidad 2 TiB y queremos montar un RAID con una capacidad neta de 10 TiB. Estamos pensando en las configuraciones de RAID 5 y RAID 6. Razona cuál sería la capacidad total que tendría cada una de estas dos configuraciones. Como sistemas RAID, indica dos características comunes y una diferencia entre un RAID 5 y un RAID 6.

Un **RAID 5** utiliza el espacio de un disco para almacenar una función de redundancia, por tanto, tendría una capacidad total de **12 TiB**, mientras que un **RAID 6** implementa dos funciones de redundancia, así que necesita el espacio de dos discos, con lo cual su capacidad total sería de **14 TiB**.

Características comunes entre un RAID 5 y un RAID 6:

- División de datos en tiras de varios bytes.
- Reparto de datos redundantes cíclicamente entre todos los discos.
- Posibilidad de varias operaciones de E/S concurrentes.

Diferencias entre un RAID 5 y un RAID 6:

- Un RAID 5 implementa una sola función de redundancia, mientras que un RAID 6 implementa dos.
- Un RAID 5 puede recuperarse de un fallo de disco, mientras que un RAID 6 puede recuperarse de dos.

[0.75p] 4. Tenemos un sistema con memoria virtual de páginas de 1 KiB que conecta la memoria principal con el almacenamiento secundario mediante un bus síncrono de 128 bits a 1 GHz.

Las transferencias de las páginas están gestionadas por una DMA a través de este bus en bloques del tamaño de la anchura del bus, necesitando 300 ns para la lectura del primer bloque y 10 ns para el resto. La DMA realiza la lectura de una página en una única transacción.

Tanto el envío de 128 bits a través del bus como el de la dirección al controlador del almacenamiento secundario requieren un ciclo. Las transferencias de datos leídos más recientemente pueden solaparse con la lectura de los siguientes.

Para finalizar la transacción la CPU atiende la interrupción enviada por la DMA ejecutando una rutina de 17 instrucciones. La CPU es multiciclo y cada instrucción necesita 4 ciclos para ejecutarse, siendo la frecuencia de CPU también de 1 GHz.

Calcula la latencia y el ancho de banda para la lectura de una página.

El ciclo de reloj del bus y de la CPU es:

$$T_{\text{ciclo}} = \frac{1}{f} = \frac{1}{1 \text{ GHz}} = 10^{-9} \text{ s} = 1 \text{ ns}$$

El sistema soporta transferencias de 16 B (128 bits). Cada página son 1 KiB, que corresponden a $1 \text{ KiB} / (16 \text{ B/transf.}) = 2^6$ transferencias = 64 transferencias en una transacción.

Para leer 1 página necesitamos:

- 1 ciclo para enviar la dirección: 1 ns.
- Un tiempo de acceso a memoria de 300 ns para leer los primeros 16 B.
- 10 ns para acceder a cada uno de los 63 siguientes bloques de 16 B que se solapan con el envío del anterior que se ha leído (1 ns).
- 1 ciclo de envío para la última palabra (1 ns).
- Al finalizar una transacción se ejecuta la rutina que atiende la interrupción de finalización, que consta de 17 instrucciones, cada una de 4 ciclos (4 ns) de duración.

Por tanto, para la lectura de una página necesitamos:

$$1 + 300 + 63 \times 10 + 1 + 17 \times 4 = 1000 \text{ ns/transacción.}$$

Así, la **latencia** para la lectura de una página es de 1000 ns, y el ancho de banda:

$$AB = \frac{1 \text{ KiB}}{1000 \times 10^{-9} \text{ s}} \simeq 10^9 \text{ B/s} = 1 \text{ GB/s}$$

Bloque B

- [2p] 5. El siguiente código se ejecuta en el procesador segmentado de 5 etapas estudiado en clase: IF, ID, EX, MEM y WB. Los saltos se deciden en la etapa ID empleando predicción de salto fijo no efectivo. Hay una unidad de detección de riesgos en la etapa ID, y una unidad de anticipación en la etapa EX. La unidad de suma en punto flotante está segmentada, y requiere 3 ciclos de reloj para efectuar una operación de suma. Cada una de las etapas MEM y WB sólo puede gestionar una instrucción en cada ciclo.

- (a) [0.5p] Muestra el diagrama multiciclo para **la primera iteración** de este código, marcando de forma explícita **las anticipaciones y los bloqueos del pipeline**.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14
1 addi \$t4, \$0, 10	IF	ID	EX	MEM	WB									
Loop:														
2 lw \$a1, 0(\$a0)		IF	ID	EX	MEM	WB								
3 add \$t0, \$a1, \$t4			IF	ID	ID	EX	MEM	WB						
4 sw \$t0, 0(\$a0)				IF	ID	ID	EX	MEM	WB					
5 addi \$t4, \$t4, -1						IF	ID	EX	MEM	WB				
6 addi \$a0, \$a0, 4							IF	ID	EX	MEM	WB			
7 bne \$t4, \$0, Loop								IF	ID	ID	EX	MEM	WB	
8 sw \$t0, 0(\$a0)									IF	IF	nop			

- (b) [0.25p] Indica dos dependencias verdaderas (de datos) y una antidependencia que aparezcan en el cuerpo del bucle.

Verdaderas: \$a1 entre la 3 y la 2 y \$t0 entre la 4 y la 3. Antidependencia: \$t4 entre la 5 y la 3

- (c) [0.25p] ¿Cuáles de estas dependencias generan un riesgo? ¿De qué tipo?

La dependencia entre la 3 y la 2 genera un riesgo RAW al tener que leer el dato de memoria en la etapa MEM de la instrucción 2 para poder anticiparlo a la etapa EX de la instrucción 3.

- (d) [0.25p] Discute la veracidad de la siguiente afirmación: “Si el procesador implementase salto retardado, se puede eliminar la instrucción 4 y, tras la ejecución, el array que recorre \$a0 tendrá los mismos valores que usando el código original en el procesador sin salto retardado, sólo que desplazados una posición.”.

Es falso. El resultado del código sería totalmente diferente, porque la instrucción 8 del hueco de retardo sobrescribirá en cada iteración el valor que leerá la instrucción 2 en la siguiente iteración.

- (e) [0.25p] Calcula la tasa de acierto del predictor de salto según los siguientes modos de procesamiento de salto que podría implementar el procesador: salto fijo no efectivo, predictor de salto de 1 bit y predictor de salto de 2 bits. En los modos de predicción dinámica, la predicción inicial es de salto no efectivo.

El salto se ejecuta 10 veces. Con salto fijo no efectivo sólo se acierta en la última ejecución: $TA = 1/10$. Con un predictor de 1 bit se fallará en la primera y la última ejecución del salto: $TA = 8/10$. Con un predictor de 2 bits se fallará en las 2 primeras y en la última ejecución: $TA = 7/10$.

- (f) [0.5p] Razona cuáles de las siguientes señales de control estarían activas (valor 1) en cualquiera de las etapas del procesado durante el ciclo de ejecución 8: MemRead, MemWrite, RegWrite, Mem2Reg, Branch, IF_flush.

WB: RegWrite (add), MEM: MemWrite (sw). El resto no se encuentran activas en el ciclo 8.

- [2p] 6. Un computador tiene una memoria principal de **256 KiB** y una memoria caché de **correspondencia directa** de **1024 B** con líneas de **4 bytes**. Las políticas de escritura son **post-escritura (write-back)** y **ubicar en escritura (write-allocate)**. Comenzando con la caché vacía, ejecutamos el siguiente código. La matriz A está almacenada por filas comenzando en la dirección 0x06000, y cada elemento de tipo “int” ocupa 4 bytes. Las variables escalares se almacenan en registros de la CPU y no generan accesos a memoria.

```
int i, j, A[7][1024];
for (i = 1; i <= 2; i++)
    for (j = 1; j <= 2; j++)
        A[j][i] = A[0][1024 - j]
```

- (a) [0.3p] Calcula la dirección de memoria ocupada por A[j][i] en función de “j” y “i”.

$$A[i][j] = 0x06000 + 0x1000 * j + 0x4 * i$$

- (b) [0.3p] Indica la partición en campos de una dirección física desde el punto de vista de la caché.

Etiqueta (8 bits), Índice (8 bits), Desplazamiento (2 bits)

- (c) [0.6p] Rellena todos los campos de la siguiente tabla de accesos a memoria: dirección, etiqueta, e índice/conjunto (en formato **hexadecimal**); indica si el acceso resulta en un acierto o un fallo caché, y en este último caso en qué tipo de fallo; e indica si el acceso provoca una escritura a memoria principal (MP) o no.

Dirección	Etiqu.	Idx	A/F	Tipo de Fallo	Escritura MP
06FFC	1B	FF	F	Forzoso	
07004	1C	1	F	Forzoso	
06FF8	1B	FE	F	Forzoso	
08004	20	1	F	Forzoso	SI
06FFC	1B	FF	A	Acierto	
07008	1C	2	F	Forzoso	
06FF8	1B	FE	A	Acierto	
08008	20	2	F	Forzoso	SI

carga 06FFC en set FF

carga+escribe 07004 en set 1

carga 06FF8 en set FE

reemplaza 07004 por 08004 (mod) en set 1

carga+escribe 07008 en set 2

reemplaza 07008 por 08008 (mod) en set 2

- (d) [0.3p] Si el tiempo medio de acceso a memoria fue de 43 ciclos, y el tiempo de acierto caché es de 1 ciclo, calcula el tiempo de acceso a memoria principal.

$$43 = 1 + \frac{6}{8} \times TMP \text{ ciclos} \rightarrow TMP = (43 - 1) \times \frac{8}{6} = 7 \times 8 = 56 \text{ ciclos}$$

- (e) [0.6p] Discute la veracidad de las siguientes afirmaciones:

- En esta caché se podría solapar la comprobación de la etiqueta con la transmisión de los datos
Sí, porque los datos sólo pueden proceder de la única línea del conjunto.
- Sólo los fallos de conflicto pueden provocar escrituras a memoria principal en esta caché.
No, cualquier fallo de cualquier tipo que expulse una línea modificada provocará una escritura a memoria.

APELLIDOS: _____ NOMBRE: _____

Bloque 1

[2p] 1. El siguiente código se ejecuta en el procesador segmentado de 5 etapas estudiado en clase: IF, ID, EX, MEM y WB. Los saltos se deciden en la etapa ID empleando predicción de salto fijo no efectivo. Hay una unidad de detección de riesgos en la etapa ID, y una unidad de anticipación en la etapa EX. La unidad de suma en punto flotante está segmentada, y requiere 3 ciclos de reloj para efectuar unaa operación de suma. Cada una de las etapas MEM y WB sólo puede gestionar una instrucción en cada ciclo.

(a) [0.4p] Muestra el diagrama multiciclo para **la primera iteración** de este código, marcando de forma explícita **las anticipaciones y los bloqueos del pipeline**.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
1 addi \$s0, \$0, 4	IF	ID	EX	MEM	WB											
Loop:																
2 lwc1 \$f1, 0(\$a1)		IF	ID	EX	MEM	WB										
3 add.s \$f2, (\$f1), \$f2			IF	ID	-	A1	A2	A3	MEM	WB						
4 addi \$a1, \$a1, 4				IF	-	ID	EX	MEM	WB							
5 addi \$s0, \$s0, -1						IF	ID	-	EX	MEM	WB					
6 bne \$s0, \$0, Loop							IF	-	ID	-	-	EX	MEM	WB		
7 swc1 \$f2, 0(\$a2)									IF	-	-	ID	nop	riesgo de control		
2 lwc1 \$f1, 0(\$a1)													IF	ID	EX	MEM

(b) [0.3p] Dentro del bucle, indica dos dependencias verdaderas (de datos) y una antidependencia.

Marcadas sobre el código: verdaderas y antidependencia

(c) [0.2p] Identifica **sobre el diagrama** un riesgo estructural y un riesgo de control.

(d) [0.3p] En este código, si el procesador emplease un **predicador de saltos dinámico de 1-bit**, razona cuántas veces se predeciría el salto de forma correcta.

2 veces: fallo, acierto, acierto, fallo (3 si se considera predicción inicial efectiva)

(e) [0.3p] Partiendo del código original, si el procesador emplease salto retardado, razona sobre la posibilidad de emplear la instrucción 5 (addi \$s0, \$s0, -1) para rellenar el hueco de retardo.

Es posible, pero desde destino de salto: hay que replicarla, o bien modificar la instrucción 1 a addi \$s0, \$0, 3. Al terminar el bucle \$s0 tendrá valor -1 en lugar de 0.

(f) [0.5p] Razona cuáles de las siguientes señales de control estarían activas en cualquiera de las etapas del procesamiento durante el quinto ciclo de ejecución: MemRead, MemWrite, RegWrite, Mem2Reg, Branch, IF flush.

WB: RegWrite (addi), MEM: MemRead (lw). El resto no se encuentran activas en el ciclo 5.

- [2p] 2. Un computador tiene una memoria principal de **1 MiB** y una memoria caché asociativa por conjuntos de **2 vías** de **512 B** con líneas de **16 bytes**, con algoritmo de reemplazo LRU. Las políticas de escritura son **post-escritura (write-back)** y **ubicar en escritura (write-allocate)**. Comenzando con la caché vacía, ejecutamos el siguiente código. La matriz A está almacenada por filas comenzando en la dirección 0x0A000, y cada elemento de tipo “int” ocupa exactamente 4 bytes. Las variables escalares se almacenan en registros de la CPU y no generan accesos a memoria.

```
int i, j, A[32][64];
for (i = 0; i < 2; i++)
    for (j = 0; j < 2; j++)
        A[j+1][i*2] = A[j][0]
```

- (a) [0.3p] Calcula la dirección de memoria ocupada por A[i][j] en función de “i” y “j”.

$$A[i][j] = 0x0A000 + 0x100i + 0x4j$$

- (b) [0.3p] Indica la partición en campos de una dirección física desde el punto de vista de la caché.

Etiqueta (12 bits), Índice (4 bits), Desplazamiento (4 bits)

- (c) [0.6p] Rellena todos los campos de la siguiente tabla de accesos a memoria: dirección, etiqueta, e índice/conjunto (en formato **hexadecimal**); indica si el acceso resulta en un acierto o un fallo caché, y en este último caso en qué tipo de fallo; e indica si el acceso provoca una escritura a memoria principal (MP) o no.

Dirección	Etiqu.	Idx	A/F	Tipo de Fallo	Escritura MP	
0A000	0A0	0	F	Forzoso		carga 0A0 en via 0
0A100	0A1	0	F	Forzoso		carga 0A1 (modificada) en via 1
0A100	0A1	0	A			
0A200	0A2	0	F	Forzoso		reemplaza 0A0 por 0A2 (mod) en via 0
0A000	0A0	0	F	Conflicto	SI	reemplaza 0A1 (mod) por 0A0 en via 1
0A108	0A1	0	F	Conflicto	SI	reemplaza 0A2 (mod) por 0A1 (mod) en via 0
0A100	0A1	0	A	Acierto		
0A208	0A2	0	F	Conflicto		reemplaza 0A0 por 0A2 (mod) en via 1

- (d) [0.3p] Si el tiempo medio de acceso a memoria fue de 43 ciclos, y el tiempo de acierto caché es de 1 ciclo, calcula el tiempo de acceso a memoria principal.

$$43 = 1 + \frac{6}{8} \times TMP \text{ ciclos} \rightarrow TMP = (43 - 1) \times \frac{8}{6} = 7 \times 8 = 56 \text{ ciclos}$$

- (e) [0.6p] Discute la veracidad de las siguientes afirmaciones:

- i. Una caché víctima reduce la tasa de fallos y, por lo tanto, también reduce el tiempo medio de acceso a memoria.

La caché víctima reduce el tiempo medio de acceso a memoria porque algunos fallos pueden resolverse en ella en lugar de acceder a MP, pero no afecta a la tasa de fallos de la caché.

- ii. En este sistema, si hubiese una caché L2 del doble de tamaño que la caché L1 y mantenido el resto de parámetros de la caché idénticos, ningún fallo provocaría una escritura a memoria principal.

Es verdadero: al duplicar el tamaño, el índice en L2 tendrá 1 bit más que en L1 y los accesos con etiqueta A01 se mapean a un conjunto diferente, por lo que los fallos de conflicto se resolverán en esa caché L2.

- iii. Si esta caché tuviese una política de escritura directa (write-through), no habría fallos de conflicto.

Falso. Sería cierto para una caché que no ubique en escritura, pero la política de escritura directa no necesariamente fuerza la política de ubicación.

APELLIDOS: _____ NOMBRE: _____

Bloque 2

[0.75p] 3. Un programa consta de 10^9 instrucciones y tarda 1.4 segundos en ejecutarse. Involucra únicamente dos subsistemas en la ejecución: CPU y E/S. El 30% de las instrucciones son sumas de valores de doble precisión, que tardan cada una exactamente 4 ciclos en ejecutarse cada una. El 10% son instrucciones de E/S que tardan de media 10 ciclos cada una. El resto de operaciones son de tipo entero y se ejecutan en 1 ciclos de reloj.

(a) Calcula el CPI.

$$\frac{10^9 \times (0.3 \times 4 + 0.1 \times 10 + 0.6 \times 1) \text{ ciclos}}{10^9 \text{ instrucciones}} = 2.8 \text{ ciclos/instr}$$

(b) Calcula la métrica FLOPS para esta ejecución.

$$\frac{0.3 \times 10^9 \text{ flops}}{1.4 \text{ s}} = 2.14 \times 10^8 = 214 \text{ MFLOPS}$$

(c) Calcula la frecuencia de reloj.

$$f_{cpu} = \frac{2.8 \times 10^9 \text{ ciclos}}{1.4 \text{ s}} = 2 \times 10^9 \text{ ciclos/s} = 2 \text{ GHz}$$

(d) En una segunda versión de esta máquina se consigue una aceleración de 2.5 en las operaciones de entrada salida. El resto de operaciones no se ven afectadas. ¿Cuánto más rápido se ejecuta el programa en esta máquina con respecto a la anterior?

$$A = \frac{10^8 \times (3 \times 4 + 1 \times 10 + 6 \times 1) \times \cancel{T_{ciclo}}}{10^8 \times (3 \times 4 + 1 \times 10/2.5 + 6 \times 1) \times \cancel{T_{ciclo}}} = \frac{28}{22} = 1.27$$

o bien

$$F_m = \frac{1 \times 10}{3 \times 4 + 1 \times 10 + 6 \times 1} = 0.357$$

$$A_m = 2.5$$

$$A = \frac{1}{1 - 0.357 + \frac{0.357}{2.5}} = 1.27$$

(e) Este programa forma parte de un benchmark SPEC. Al ejecutarlo 3 veces se han medido tiempos de 1.3, 1.1 y 1.6 segundos. La máquina de referencia para ese benchmark ha tardado 24 segundos en ejecutarlo. Calcula el ratio SPEC individual para este programa.

Para el cálculo del ratio SPEC se utiliza el valor mediano: 1.3 segundos.

$$\frac{24 \text{ s}}{1.3 \text{ s}} = 18.46$$

[2p] 4. Un computador con 1 GiB (2^{30} bytes) de memoria principal utiliza memoria virtual paginada con traducción en 2 niveles. El tamaño de página es de 4 KiB (2^{12} bytes). Cada tabla de páginas involucrada en el proceso de traducción ocupa exactamente una página física, y cada entrada comienza con 1 bit de residencia, seguido de 1 bit de modificación, 12 bits adicionales de protección y control, y el número de página física.

(a) Determina el tamaño del espacio virtual.

Una DF, de 30 bits, se divide en 12 bits de desplazamiento y 18 bits de página física. Cada entrada de la tabla de páginas ocupa $1 + 1 + 12 + 18 = 32$ bits = 4 bytes. Una tabla de páginas, que ocupa 2^{12} bytes, contiene $2^{12}/4 = 2^{10}$ entradas. La DV se dividirá en 10 bits PV1, 10 bits PV2 y 12 bits Despl. Por tanto, la DV tiene 32 bits y el espacio virtual es de 2^{32} bytes = 4 GiB.

(b) En la traducción de la dirección virtual `0x0116453C`, la entrada correspondiente de la tabla de páginas de último nivel (la última tabla consultada antes de terminar el proceso) contiene el valor `0x90431F45`. Razona si se puede hacer una traducción válida y calcula la dirección física resultante.

El desplazamiento de la DV es `0x53C`. El bit de residencia de la entrada es 1 (`0x9 = 1001`), por lo que la traducción se puede realizar. El número de PF son los últimos 18 bits de la entrada: `0x31F45`. La DF resultante es `0x31F4553C`.

(c) La siguiente tabla muestra el contenido de la TLB de un proceso. Indica una dirección virtual que pueda ser traducida a través de esta TLB, y cuál sería la dirección física correspondiente.

PV	PF
<code>0x44510</code>	<code>0x1F34C</code>
<code>0xD178C</code>	<code>0x2ABD0</code>
<code>0x44511</code>	<code>0x0A1B3</code>
<code>0x3A022</code>	<code>0x1BD42</code>

Si escogemos por ejemplo la primera entrada de la TLB y utilizamos un desplazamiento arbitrario, como por ejemplo 0, la DV `0x44510000` se traduciría en la DF `0x1F34C000`.

(d) Describe brevemente una utilidad de la memoria virtual

Ejemplo de respuestas posibles: (i) ejecución de programas más grandes que la memoria física, (ii) aislamiento de procesos, (iii) ejecución de múltiples programas que en conjunto ocupan más que la memoria física disponible, (iv) flexibilidad en la asignación de memoria

(e) Indica una ventaja y un inconveniente de una caché virtual con respecto a una caché física en la gestión de la memoria virtual

Ejemplo de respuesta: no es necesario traducir direcciones para acceder a una caché virtual porque las etiquetas e índices se obtienen de la dirección virtual, pero es necesario implementar un mecanismo para garantizar la seguridad y el aislamiento entre procesos, como por ejemplo incluir el PID del proceso en el directorio caché o reemplazar todo el contenido de la caché en cada cambio de contexto.

APELLIDOS: _____ NOMBRE: _____

Bloque 3

[0.5p] 5. Completa correctamente las siguientes afirmaciones:

- (a) Suponiendo discos de 1 TiB y una capacidad neta de almacenamiento de 8 TiB, un **RAID 6** tendrá una capacidad de almacenamiento total de **10 TiB**.
- (b) Algunas de las ventajas de los discos de estado sólido **SSD** frente a los discos duros magnéticos **HDD** son ser **más rápidos** y más silenciosos.
- (c) Los discos duros (**HDD**) y las unidades de estado sólido (**SSD**) forman parte del nivel de memoria **secundaria** de un computador.
- (d) Los **RAID 4** y **5** son similares, pero el **RAID 5** no tiene el problema de cuello de botella en el acceso a la información de paridad.
- (e) Cuando queremos implementar un **RAID**, si utilizamos un único modelo de disco de un fabricante, es aconsejable que sean de **distinta serie de fabricación** para aumentar la fiabilidad.

[0.75p] 6. Tenemos un sistema con memoria virtual de páginas de 4 KiB que conecta la memoria principal con el almacenamiento secundario mediante un bus síncrono de 64 bits a 50 MHz.

Las transferencias de las páginas están gestionadas por una DMA a través de este bus en transacciones de 128 palabras (8 bytes/palabra), necesitando 300 ns para la lectura de la primera palabra y 150 ns para el resto.

Tanto el envío de 64 bits a través del bus como el de la dirección al controlador del almacenamiento secundario requieren un ciclo. Las transferencias de datos leídos más recientemente pueden solaparse con la lectura de los siguientes. Entre transacciones de bus es necesario esperar 2 ciclos de reloj de bus.

Calcula la latencia y el ancho de banda para la lectura de una página.

$$T_{\text{ciclo_bus}} = \frac{1}{f} = \frac{1}{50 \text{ MHz}} = \frac{1}{5} 10^{-7} \text{ s} = 20 \text{ ns}$$

El sistema soporta transacciones de 128 palabras. Cada página son 4 KiB, que corresponden a $4 \text{ KiB} / (8 \text{ B/p}) = 2^9$ palabras. Por tanto, para transferir una página, se requieren: $2^9 \text{ p} / (128 \text{ p/transacción}) = 4$ transacciones.

Para leer las 128 palabras de 1 transacción necesitamos:

- 1 ciclo para enviar la dirección: 20 ns.
- Un tiempo de acceso a memoria de 300 ns para leer la primera palabra.
- 150 ns para acceder a cada una de las siguientes palabras que se solapa con el envío de la anterior palabra leída (20 ns).
 - Como cada transacción es de 128 palabras necesitamos 127 lecturas adicionales (el tiempo de lectura es superior al tiempo de envío).
- 1 ciclo de envío para la última palabra.
- Al finalizar una transacción hay que esperar 2 ciclos de reloj para empezar con la siguiente: 40 ns entre transacciones.

Por tanto, para la lectura de las 128 palabras de 1 transacción necesitamos:

$$200 + 300 + 127 \times 150 + 20 = 19390 \text{ ns/transacción.}$$

La latencia para la lectura de una página será:

$$4 \text{ transacciones} \times 19390 \text{ ns/transacción} + 3 \text{ esperas} \times 40 \text{ ns/espera} = \boxed{77680 \text{ ns} \simeq 77.7 \mu\text{s}}$$

$$\text{Y el ancho de banda será: } 4 \text{ KiB} / (77680 \times 10^{-9} \text{ s}) \simeq \boxed{50.3 \text{ MiB/s} \simeq 52.7 \text{ MB/s}}$$

ESTRUCTURA DE COMPUTADORES

21 de junio de 2023

Normas: Escribe los apellidos y el nombre en cada hoja de examen y en cada hoja adicional que utilices. Es posible escribir la respuesta en la misma hoja de cada ejercicio, pero si utilizas hojas adicionales no mezcles en la misma hoja respuestas de bloques diferentes: se entregará cada bloque por separado.

- [0.75p] 1. Un *benchmark* en un procesador determinado tarda 5 s en finalizar, con un total de 20×10^9 instrucciones ejecutadas. Sabemos que en este *benchmark* el 25% del tiempo se emplea en la ejecución de instrucciones de punto flotante. Contesta razonadamente estas cuestiones:

- (a) [0.2p] ¿Cuál es el MIPS alcanzado?

$$MIPS = \frac{\text{Num. instrucciones}}{\text{Tempo de ejecución (s)} \times 10^6} = \frac{2 \times 10^{10}}{5 \times 10^6} = 4000$$

- (b) [0.2p] ¿Cuál es el GFLOPS alcanzado?

$$GFLOPS = \frac{\text{Num. instrucciones P. F.}}{\text{Tempo de ejecución (s)} \times 10^9} = \frac{2 \times 10^{10} \times 0.25}{5 \times 10^9} = 1$$

- (c) [0.35p] Aplicando la ley de Amdhal, ¿cuánto tendríamos que mejorar la unidad de punto flotante si queremos conseguir una aceleración del 10%?

Ley de Amdhal:

$$A = \frac{1}{(1 - F_{PF}) + \frac{F_{PF}}{A_{PF}}}$$

Temos $A = 1.1$ e $F_{PF} = 0.25$ co cal:

$$1.1 = \frac{1}{(1 - 0.25) + \frac{0.25}{A_{PF}}}$$

Por tanto:

$$A_{PF} = \frac{0.25 \times 1.1}{1 - 0.75 \times 1.1} \approx 1.57$$

Teríamos que mellorar a unidade de punto flotante un 57%.

- [2.0p] 2. El siguiente código se ejecuta en el procesador segmentado MIPS de 5 etapas estudiado en clase: IF, ID, EX, MEM e WB. El salto se decide en la etapa ID, se usa la técnica de salto fijo no efectivo, tiene una unidad de detección de riesgos en la etapa ID y una unidad de anticipación en la etapa EX. La ejecución de una operación de suma en la unidad de punto flotante requiere 3 ciclos y está segmentada. Las etapas MEM e WB solo pueden albergar una instrucción.

```
1  addi $s0, $0, 4
   Loop:
2  lwc1 $f1, 0($a1)
3  add.s $f2, $f1, $f2
4  addi $a1, $a1, 4
5  addi $s0, $s0, -1
6  bne $s0, $0, Loop
   End:
7  swc1 $f2, 0($a2)
```

Contesta **razonadamente** las siguientes cuestiones:

- (a) [0.2p] Indica dos dependencias verdaderas que haya dentro del bucle.

- Entre as instrucións 2 e 3, xa que na 2 `lwc1` modifica `$f1` e a continuación na 3 `add.s` utiliza ese rexistro.
- Entre as instrucións 5 e 6, xa que na 5 `addi` modifica `$s0` e a continuación na 6 `bne` compara ese valor con 0.

- (b) [0.1p] Indica una antidependencia que haya dentro del bucle.

Entre as instrucións 2 e 4, xa que na 2 `lwc1` le a posición de memoria a partir do enderezo almacenado en `$a1` e na 4 a `addi` modifica ese valor.

- (c) [0.2p] ¿Se produce alguna anticipación en la ejecución de este código? En caso afirmativo indica entre qué instrucciones y etapas.

Si, entre as instrucións 2 e 3. A instrución 3 (`add.s`) ten que esperar a que a 2 (`lwc1`) lea o valor de memoria, pero antes de que se escriba en `$f1` na etapa WB xa se pode ler na etapa EX desde o rexistro de segmentación MEM/WB.

- (d) [0.3p] Indica los riesgos de datos que se producen en el bucle del código y de qué tipo son.

Hai dous de tipo RAW (lectura despois de escritura):

- Entre as instrucións 2 e 3 pola escritura de `$f1` en 2 e a lectura en 3.
- Entre as instrucións 5 e 6 pola escritura de `$s0` en 5 e a lectura en 6.

- (e) [0.3p] ¿Cómo se podría optimizar la ejecución del código reordenando las instrucciones 3, 4 y 5?

Poñendo a instrución 5 despois da 2 evítase o bloqueo entre a 2 e a 3 e tamén o que se produce entre a 5 e a 6 debido aos riscos de datos, conseguindo executar o código en menos ciclos. Quedaría:

```
1 addi $s0, $0, 4
2 Loop: lwc1 $f1, 0($a1)
5 addi $s0, $s0, -1
3 add.s $f2, $f1, $f2
4 addi $a1, $a1, 4
6 bne $s0, $0, Loop
7 End: swc1 $f2, 0($a2)
```

- (f) [0.5p] Partiendo del código original, si utilizásemos salto retardado en el lugar de salto fijo no efectivo, ¿qué instrucción colocarías en el hueco de retardo?

Podería colocarse tanto a 3, como a 4, como a 5:

- Se poñemos a 3 evitamos o bloqueo con 2, que é dun ciclo, e o código se executa correctamente. Crea un risco RAW de 2 ciclos á saída do bucle coa instrución 7.
- Se poñemos a 4 non evitamos ningún bloqueo e creamos un risco de datos coa 2 no cambio de iteración.
- Se poñemos a 5 temos que modificar o valor inicial en 1: `addi $0, $0, 3`. Evítase o bloqueo entre 5 e a 6, que demora 2 ciclos.
- Se deixamos a 7, o código funcionaría igual, pero non se melloraría o rendemento.
- Se poñemos a 2, habería que duplicala antes de entrar no bucle e podería dar un erro na iteración final por acceso a unha posición non válida.

A mellor solución é poñer a 5 porque reduce a latencia de cada iteración do bucle en 2 ciclos.

- (g) [0.4p] ¿Qué etapas de **manera efectiva** son utilizadas por la instrucción `bne`? Si el salto se decidiese en la etapa EX, ¿cuáles serían las etapas que esta instrucción usaría?

Como o salto se decide na etapa ID, a `bne` só utiliza as etapas IF, para cargar de memoria a instrución, e a etapa ID, onde se comparan os rexistros e se calcula a dirección de salto, que se fai efectivo se os valores dos rexistros son distintos.

No caso de que o salto se decidise na etapa EX, é nesta etapa onde se calcula a dirección de salto e onde se fai a comparación cunha resta. Logo na etapa MEM se activa o salto modificando o valor de PC no caso de que a comparación feita na etapa anterior indicase que os rexistros teñen valores distintos. Así neste caso a `bne` utilizaría de maneira efectiva as etapas IF, ID, EX e MEM.

- [2p] 3. Un sistema con 1 MiB de memoria principal incluye una caché L1 de 32 KiB, asociativa por conjuntos de 2 vías, con líneas de 16 B. El sistema de memoria sigue una política de postescritura. Estando la caché inicialmente vacía, se ejecuta el siguiente código.

```
int i, A[256], B[128];
for (i = 0; i < 4; i++)
    B[i] = A[i+1] + A[i+128];
```

El vector **A** se encuentra almacenado a partir de la dirección de memoria 0x0A000, mientras que **B** se encuentra almacenado inmediatamente a continuación de **A**. El resto de variables no provocan accesos a memoria caché. Cada elemento de tipo **int** ocupa exactamente 4 bytes.

- (a) [0.3p] Calcula las direcciones de memoria a las que accede este código.

$\&A = 0x0A000$

$\&B = \&A + 256 \times 4 = 0x0A000 + 0x400 = 0x0A400$

- | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
|--|-----------|-------------|-----------|----------|-----------|-----------|--------|-----------|-------------|--------|-----------|-----------|----------|-----------|-----------|--------|-----------|-------------|--|--------|-----------|-----------|----------|-----------|-----------|--------|-----------|-------------|--------|-----------|-----------|----------|-----------|-----------|--------|-----------|-------------|
| <ul style="list-style-type: none"> Iteración $i = 0$: <table border="0"> <tr><td>$A[1]$</td><td>$0x0A004$</td><td>(lectura)</td></tr> <tr><td>$A[128]$</td><td>$0x0A200$</td><td>(lectura)</td></tr> <tr><td>$B[0]$</td><td>$0x0A400$</td><td>(escritura)</td></tr> </table> Iteración $i = 1$: <table border="0"> <tr><td>$A[2]$</td><td>$0x0A008$</td><td>(lectura)</td></tr> <tr><td>$A[129]$</td><td>$0x0A204$</td><td>(lectura)</td></tr> <tr><td>$B[1]$</td><td>$0x0A404$</td><td>(escritura)</td></tr> </table> | $A[1]$ | $0x0A004$ | (lectura) | $A[128]$ | $0x0A200$ | (lectura) | $B[0]$ | $0x0A400$ | (escritura) | $A[2]$ | $0x0A008$ | (lectura) | $A[129]$ | $0x0A204$ | (lectura) | $B[1]$ | $0x0A404$ | (escritura) | <ul style="list-style-type: none"> Iteración $i = 2$: <table border="0"> <tr><td>$A[3]$</td><td>$0x0A00C$</td><td>(lectura)</td></tr> <tr><td>$A[130]$</td><td>$0x0A208$</td><td>(lectura)</td></tr> <tr><td>$B[2]$</td><td>$0x0A408$</td><td>(escritura)</td></tr> </table> Iteración $i = 3$: <table border="0"> <tr><td>$A[4]$</td><td>$0x0A010$</td><td>(lectura)</td></tr> <tr><td>$A[131]$</td><td>$0x0A20C$</td><td>(lectura)</td></tr> <tr><td>$B[3]$</td><td>$0x0A40C$</td><td>(escritura)</td></tr> </table> | $A[3]$ | $0x0A00C$ | (lectura) | $A[130]$ | $0x0A208$ | (lectura) | $B[2]$ | $0x0A408$ | (escritura) | $A[4]$ | $0x0A010$ | (lectura) | $A[131]$ | $0x0A20C$ | (lectura) | $B[3]$ | $0x0A40C$ | (escritura) |
| $A[1]$ | $0x0A004$ | (lectura) | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| $A[128]$ | $0x0A200$ | (lectura) | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| $B[0]$ | $0x0A400$ | (escritura) | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| $A[2]$ | $0x0A008$ | (lectura) | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| $A[129]$ | $0x0A204$ | (lectura) | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| $B[1]$ | $0x0A404$ | (escritura) | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| $A[3]$ | $0x0A00C$ | (lectura) | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| $A[130]$ | $0x0A208$ | (lectura) | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| $B[2]$ | $0x0A408$ | (escritura) | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| $A[4]$ | $0x0A010$ | (lectura) | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| $A[131]$ | $0x0A20C$ | (lectura) | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| $B[3]$ | $0x0A40C$ | (escritura) | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |

- (b) [0.3p] Determina cómo se divide una dirección física desde el punto de vista de la caché.

Caché de 32KiB, líneas de 16B, asociativa por conjuntos de 2 vías

- Tamaño de línea: $16\ B = 2^4\ B \rightarrow 4\ bits\ desplazamiento$
- Número de líneas: $\#L = \frac{32\ KiB}{16\ B/linea} = \frac{2^{15}\ B}{2^4\ B/linea} = 2^{11} lineas$
- Número de conjuntos: $\frac{2^{11}\ lineas}{2\ vias/conjunto} = 2^{10} conjuntos \rightarrow 10\ bits\ indice$
- Etiqueta: 6 bits (bits restantes hasta 20)

Dirección física		
Etiqueta	Índice	Desplazamiento
6 bits	10 bits	4 bits

- (c) [0.4p] Rellena los campos de la siguiente tabla para los accesos a memoria caché: dirección, etiqueta e índice/conjunto (en **hexadecimal**); si el acceso es un acierto o un fallo; en este último caso, qué tipo de fallo; y si se produce una escritura en memoria principal (MP WR).

Dirección	Etiqueta	Conjunto	Acierto/Fallo	Tipo de fallo	MP WR
0x0A004	0x02	0x200	F	Forzoso	—
0x0A200	0x02	0x220	F	Forzoso	—
0x0A400	0x02	0x240	F	Forzoso	N
0x0A008	0x02	0x200	A	—	—
0x0A204	0x02	0x220	A	—	—
0x0A404	0x02	0x240	A	—	N
0x0A00C	0x02	0x200	A	—	—
0x0A208	0x02	0x220	A	—	—
0x0A408	0x02	0x240	A	—	N
0x0A010	0x02	0x201	F	Forzoso	—
0x0A20C	0x02	0x220	A	—	—
0x0A40C	0x02	0x240	A	—	N

- (d) [0.3p] Indica cómo quedaría el directorio caché en las líneas ocupadas tras la ejecución de este código. BV = Bit de validez, BM = Bit de modificación

Conjunto	Vía	BV	BM	Etiqueta
0x200	0	1	0	0x02
0x201	0	1	0	0x02
0x220	0	1	0	0x02
0x240	0	1	1	0x02

- (e) [0.3p] Sabiendo que el tiempo medio de acceso a memoria durante la ejecución del apartado anterior fue de 52 ciclos, y que el tiempo de acierto caché es de 1 ciclo, calcula el tiempo medio de acceso al nivel inferior.

El tiempo medio de acceso al nivel inferior es la penalización por fallo en este nivel.

- 12 accesos $\rightarrow$ 8 aciertos, 4 fallos.
- $T = 52 \text{ ciclos} = 1 \text{ ciclo/acierto} + 4/12 \text{ fallos} \times P_{fallo}$
- $P_{fallo} = (52 - 1) \times \frac{12}{4} = 51 \times 3 = \boxed{153 \text{ ciclos}}$

- (f) [0.4p] Calcula el tamaño mínimo de la caché L1 tal que, manteniendo constante el resto de su configuración (tamaño de línea, asociatividad), preserve la tasa de fallos observada.

Se producirá un conflicto en el momento en que las líneas que en este caso se asignan a los conjuntos 0x200 y 0x240 se asignen al mismo conjunto, es decir, en el momento en que el n^o de conjuntos disminuya a 0x40 $\rightarrow$ Tamaño caché = 0x40 $\times$ 2 vías/conjunto $\times$ 16B/línea = 2 KiB.

- [2p] 4. Un sistema cuenta con 64 GiB de memoria principal (2^{36} B) y un espacio virtual paginado de 1 TiB (2^{40} B) con un esquema de traducción en 2 niveles. El tamaño de página es de 64 KiB (2^{16} B). Cada tabla de páginas de primer o segundo nivel tiene 4096 entradas (2^{12}) de 4 bytes cada una, donde el bit más significativo de cada entrada es el bit de residencia o validez, y el número de página física está codificado en los bits menos significativos. Los bits restantes son bits de control. La CPU solicita los datos correspondientes a la dirección virtual 0x01 F004 D38C. En la entrada de la tabla de páginas de primer nivel se encuentra el contenido 0xA274 14A0 y en la entrada de la tabla de páginas de segundo nivel 0x8002 4605.

- (a) **0.3p** Determina en qué campos (longitud y valores) se divide la dirección virtual.

El tamaño de página es 2^{16} bytes, por lo que se necesitan 16 bits para el desplazamiento. Cada TP de primer o segundo nivel tiene 2^{12} entradas. Entonces son 12 bits para PV1, 12 bits para PV2 y 16 bits para desplazamiento.

Dirección virtual		
numPV1	numPV2	Δ_p
12 bits	12 bits	16 bits
0x01F	0x004	0xD38C

- (b) **0.3p** Calcula la dirección física de la entrada correspondiente de la tabla de páginas de nivel 2.

Del contenido de la entrada de la TP1, que está en el enunciado: 0xA27414A0, comprobamos el bit de residencia: comienza por 0xA, en binario 1010, y obtenemos la página física en la que se encuentra la tabla de nivel 2 (los 20 bits menos significativos): 0x414A0. El registro base de la tabla es la dirección física en la que comienza esa tabla: 0x4 14A0 0000. Partiendo de esa dirección desplazamos 4 bytes/entrada hasta la entrada 0x004, correspondiente al número de PV de nivel 2.

- $\text{DirE2} = 0x4\ 14A0\ 0000 + 0x004 \times 4 = 0x4\ 14A0\ 0010$

- (c) **0.4p** ¿Cuál es la dirección física resultante del proceso de traducción?

La página física son los 20 bits menos significativos de la entrada de la TP2: 0x24605. También es necesario comprobar el bit de residencia: comienza por 0x8 $\rightarrow$ 1000. A esa página física le concatenamos el desplazamiento 0xD38C.

Dirección física	
numPF1	Δ_p
20 bits	16 bits
0x24605	0xD38C

La dirección física resultante es 0x2 4605 D38C.

- (d) **0.4p** Calcula cuánto ocupan las tablas de páginas que es necesario que residan en memoria física para realizar esta traducción

Es suficiente con que resida la TP de primer nivel y una TP de segundo nivel. Cada tabla ocupa $2^{12} \times 4 \text{ bytes} = 2^{14} \text{ bytes} = 16 \text{ KiB}$. Por tanto, las tablas necesarias para la traducción ocupan $2 \times 16 \text{ KiB} = 32 \text{ KiB}$

- (e) **0.2p** Determina si existe fragmentación interna o externa en este sistema en la gestión de la memoria virtual (pista: puedes utilizar los cálculos del apartado anterior).

Cada TP hemos calculado que ocupa 16 KiB , pero el tamaño de página es de 64 KiB . Por lo tanto, habrá fragmentación interna porque de cada página utilizada en el proceso de traducción sólo se utilizará la cuarta parte.

No existe fragmentación externa en un sistema paginado.

- (f) **0.4p** La caché L1 es de 32 KiB , asociativa por conjuntos de 8 vías con un tamaño de línea de 64 bytes. Determina si podría ser una caché con índices virtuales y etiquetas físicas (VIPT).

Las direcciones caché en L1 tienen 6 bits de desplazamiento (el tamaño de línea es de 64 bytes) y 6 bits de índice (512 líneas en conjuntos de 8 vías $\rightarrow$ 64 conjuntos).

Para implementar la caché VIPT es necesario y suficiente que los bits de desplazamiento (6) e índice (6) de la dirección física coincidan dentro de los bits de desplazamiento de la dirección virtual (16). Como $6 + 6 < 16$, la caché sí podría ser VIPT.

[0.5p] 5. Disponemos de 4 discos de 1 TB en una configuración RAID 0.

- (a) **0.3p** ¿Cuántos discos necesitaríamos para mantener en el sistema la misma cantidad de información neta pero si lo quisiéramos configurar como RAID 4, 5 o 6?

RAID 0 no tiene redundancia, por lo que tiene 4 TB de información neta.

- RAID 4 tiene un disco adicional para mantener información de paridad: 5 discos en total.
- RAID 5 también mantiene información redundante equivalente a un disco, pero distribuida por todo el conjunto: 5 discos en total.
- RAID 6 tiene doble paridad distribuida, de tamaño equivalente a 2 discos: 6 discos en total.

- (b) **0.2p** ¿Qué ventajas o inconvenientes tendría utilizar RAID 5 en este sistema sobre RAID 4?

La paridad en RAID 5 está distribuida, lo que evita un cuello de botella al acceder constantemente el disco de paridad de RAID 4 cada vez que se realiza una escritura. Esto también mejora el rendimiento de lecturas/escrituras concurrentes.

En cuanto a fiabilidad, ambos pueden recuperarse ante el fallo de un único disco cualquiera, reconstruyendo su información gracias al resto de información junto con la información de paridad. RAID 4 puede considerarse menos fiable porque el disco de paridad se escribe con mayor frecuencia (en cada escritura), lo que puede reducir su vida útil.

[0.75p] 6. Sea un computador con las siguientes características:

- Un sistema de memoria y de bus que soporta el acceso a bloques de 64 palabras de 32 bits cada una.
- Un bus síncrono de 64 bits a 2 GHz, en el que tanto una transferencia de 64 bits como el envío de una dirección de memoria requieren 1 ciclo de reloj.
- El tiempo de acceso a memoria para cada 4 palabras es de 1 ns.
- Las transferencias por el bus y los accesos a memoria pueden solaparse. Se supone que el bus está disponible antes de cada acceso.

Calcula la latencia y el ancho de banda para la lectura de 1024 palabras.

$$T_{ciclo} = \frac{1}{f} = \frac{1}{2GHz} = \frac{1}{2}10^{-9} s = 0.5 ns$$

El sistema soporta bloques de 64 palabras. Por tanto, para transferir 1024 palabras, se requieren $1024 p / (64 p/transaccion) = 16$ transacciones.

Para leer las 64 palabras de 1 transacción necesitamos:

- 1 ciclo para enviar la dirección.
- Un tiempo de acceso a memoria de $1 ns / (0.5 ns/ciclo) = 2$ ciclos para leer las 4 primeras palabras.
- 2 ciclos para enviar los datos del grupo de 4 palabras leído (el bus es de 64 bits, por lo que enviamos 2 palabras por ciclo), que se solapa con el tiempo de acceso del siguiente grupo de 4 palabras.
 - Como cada uno de estos grupos está formado por 4 palabras, necesitamos repetir 16 veces para transferir las 64 palabras que forman 1 transacción.

Por tanto, para la lectura de las 64 palabras de 1 transacción necesitamos $1 + 2 + 16 \times 2 = 35$ ciclos/transaccion

La latencia para la lectura de 1024 palabras será $16 transacciones \times 35 ciclos/tr \times 0.5 ns/ciclo =$
280 ns

Y el ancho de banda será $(1024 palabras \times 4 B/palabra) / (136 \times 10^{-9} s) = 14.63 \times 10^9 B/s =$
14.63 GB/s

APELLIDOS: _____ NOMBRE: _____

Hojas adicionales adjuntas: _____

Normas: Escribe los apellidos y el nombre en cada hoja de examen y en cada hoja adicional que utilices. Es posible escribir la respuesta en la misma hoja de cada ejercicio, pero si utilizas hojas adicionales no mezcles en la misma hoja respuestas de ejercicios diferentes: se entregarán los ejercicios de cada hoja por separado. La respuesta a los ejercicios debe estar escrita con bolígrafo. Al terminar, indica en cada hoja de examen las hojas adicionales utilizadas.

- [3p] 1. El siguiente fragmento de código se ejecuta en un procesador segmentado de 5 etapas como el MIPS estudiado en clase: IF, ID, EX, MEM e WB. En ninguna de las etapas pueden coincidir dos instrucciones, aunque utilicen bancos de registros diferentes. El procesador tiene una frecuencia de reloj de 2 GHz, una unidad de detección de riesgos en la etapa ID y una unidad de anticipación en la etapa EX. La ejecución de una suma en punto flotante tiene una latencia de 2 ciclos. El salto se decide en la etapa ID y el procesador usa la técnica de salto fijo no efectivo.

```

1. addi $s3, $0, 4
2. loop: lwc1 $f0, 0($s2)
3.     add.s $f1, $f0, $f0
4.     add.s $f2, $f2, $f1
5.     addi $s3, $s3, -1
6.     add $s2, $s2, 4
7.     bne $s3, $0, loop
8. swc1 $f2, 0($s2)

```

- (a) Dibuja el diagrama en múltiples ciclos para la ejecución de una iteración del código. Señala **explícitamente** las anticipaciones y los bloqueos. (1.0p)

En **color verde** se destacan los ciclos que forman parte del bucle. En **color rojo**, los bloqueos y anticipaciones.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
1	IF	ID	EX	MEM	WB												
2		IF	ID	EX	MEM	WB ↓\$f0											
3			IF	ID	(ID)	A1	A2	MEM ↓\$f1	WB								
4				IF	(IF)	ID	(ID)	A1	A2	MEM	WB						
5						IF	(IF)	ID	EX	(EX)	MEM	WB					
6								IF	ID	(ID)	EX	MEM	WB				
7									IF	(IF)	ID	(ID)	EX	MEM	WB		
8											IF	(IF)	(nop)	(nop)	(nop)	(nop)	
2													IF	ID	EX	MEM	WB
8											IF	ID	EX	MEM	WB		

El bloqueo en el ciclo 10 para prevenir el riesgo estructural podría adelantarse al ciclo 9 si consideramos que lo produce la unidad de detección de riesgos en la etapa ID. Ambas consideraciones son igualmente válidas.

- (b) Indica sobre el diagrama de qué tipo son los riesgos presentes en la ejecución. (0.2p)

- 2→3 (ciclo 5), 3→4 (ciclo 7), 5→7 (ciclo 12) Riesgo RAW
- 4→5 (ciclo 10) Riesgo estructural
- 7→8 (ciclo 13) Riesgo de control

(c) Calcula el CPI para la ejecución completa de este código. (0.3p)

$$\begin{aligned}\#ciclos &= 5 \text{ ciclos iniciales} + 11 \text{ ciclos/iteracion} \times 4 \text{ iteraciones} = 49 \text{ ciclos} \\ \#instrucciones &= 1 \text{ instr inicial} + 6 \text{ instr/iteracion} \times 4 \text{ iteraciones} + 1 \text{ instr final} = 26 \text{ instr} \\ CPI &= \frac{49 \text{ ciclos}}{26 \text{ instr}} = 1.88 \text{ ciclos/instruccion}\end{aligned}$$

(d) Calcula el rendimiento según la métrica MFLOPS. (0.5p)

$$\begin{aligned}\#flops &= 2 \text{ flops/iteracion} \times 4 \text{ iteraciones} = 8 \text{ instrucciones} \\ T_{cpu} &= ciclos \times T_{ciclo} = \frac{ciclos}{f} = \frac{49}{2 \times 10^9} = 24.5 \text{ ns} \\ MFLOPS &= \frac{flops}{T_{cpu} \times 10^6} = \frac{8}{24.5 \times 10^{-9} \times 10^6} = \frac{8}{24.5} \times 10^3 = 326.53 \text{ MFLOPS}\end{aligned}$$

(e) Supón que el código se ejecuta en un procesador que implementa la técnica de salto retardado. Justifica, para cada una de las instrucciones, si la podrías mover o no al hueco de retardo (indicando también posibles modificaciones necesarias), y qué efecto tendría sobre los riesgos de la ejecución (es decir, si se crea o se elimina algún riesgo). **Indica razonadamente cuál es la mejor opción.** (1.0p)

1	<code>addi \$s3, \$0, 4</code>	No es posible: inicializa el iterador, tiene una dependencia de datos con 5 y además crearía un bucle infinito.
2	<code>lwc1 \$f0, 0(\$s2)</code>	Sí sería posible, duplicándola antes del bucle. El riesgo con la instrucción 2 se mantendría por lo que no supone ninguna mejora.
3	<code>add.s \$f1, \$f0, \$f0</code>	No es posible, por sus dependencias con 2 y 4.
4	<code>add.s \$f2, \$f2, \$f1</code>	Sí es posible. Elimina el riesgo con 3 pero crea uno con 8 al salir del bucle.
5	<code>addi \$s3, \$s3, -1</code>	Sí, pero habría que cambiar el inmediato de 1 por '3' o bien duplicar esta instrucción antes del bucle. Elimina el riesgo con 7. Es la mejor opción
6	<code>add \$s2, \$s2, 4</code>	Sí es posible pero se incrementaría en un ciclo el riesgo de 5 con 7.
8	<code>swc1 \$f2, 0(\$s2)</code>	No es posible, porque sobrescribiría todas las posiciones del array que recorre \$s2.

APELLIDOS: _____ NOMBRE: _____

Hojas adicionales adjuntas: _____

- [2p] 2. Un computador tiene una memoria principal de 4 GiB y una memoria caché de 64 KiB con un tamaño de línea de 16 bytes. Estando la caché inicialmente vacía, se ejecuta un código que accede a determinadas direcciones de memoria en el orden indicado en las siguientes tablas. Todos los accesos son de escritura, y suponemos que la caché utiliza una política de ubicar en escritura (allocate-on-write). Rellena los campos de las siguientes tablas: etiqueta, índice / conjunto y desplazamiento dentro de línea caché de cada dirección; si el acceso es un acierto o un fallo; en este último caso, qué tipo de fallo; y si el acceso provoca una escritura en Memoria Principal (MP) o no.

- (a) La memoria caché es de correspondencia directa. La política de escritura es de postescritura (write-back) (0.75p)

Interpretación de una dirección física desde el punto de vista de la caché:

Etiqueta	Índice	Desplazamiento
16 bits	12 bits	4 bits

Procesamiento de direcciones:

Dirección	Etiqu.	Índice	Despl.	Acierto/Fallo	Tipo de fallo	Escritura MP?
0xFF306423	FF30	642	3	Fallo	Forzoso	No
0xFF30642A	FF30	642	A	Acierto	—	No
0xFF30642F	FF30	642	F	Acierto	—	No
0xFF3A6423	FF3A	642	3	Fallo	Forzoso	Sí
0xFF3A6427	FF3A	642	7	Acierto	—	No
0xFF306424	FF30	642	4	Fallo	Conflicto	Sí

- (b) La memoria caché es asociativa por conjuntos de 16 vías. El algoritmo de reemplazo utilizado es LRU (Least Recently Used). La política de escritura es de escritura directa (write-through) (0.75p)

Interpretación de una dirección física desde el punto de vista de la caché:

Etiqueta	Conjunto	Desplazamiento
20 bits	8 bits	4 bits

Procesamiento de direcciones:

Dirección	Etiqu.	Índice	Despl.	Acierto/Fallo	Tipo de fallo	Escritura MP?
0xFF306423	FF306	42	3	Fallo	Forzoso	Sí
0xFF30642A	FF306	42	A	Acierto	—	Sí
0xFF30642F	FF306	42	F	Acierto	—	Sí
0xFF3A6423	FF3A6	42	3	Fallo	Forzoso	Sí
0xFF3A6427	FF3A6	42	7	Acierto	—	Sí
0xFF306424	FF306	42	4	Acierto	—	Sí

- (c) Sabiendo que el tiempo medio de acceso a memoria durante la ejecución del apartado b) fue de 34 ciclos, y que el tiempo de acierto caché es de 1 ciclo, calcúlese el tiempo de acceso a memoria principal. **(0.5p)**

$$T_m = T_a + P_f \cdot F_f = 34 \text{ ciclos}$$

$$1 \text{ ciclo} + P_f \cdot \frac{1}{3} = 34 \text{ ciclos} \Rightarrow P_f = 99 \text{ ciclos}$$

El tiempo de acceso a memoria es de (1+99) ciclos.

APELLIDOS: _____ NOMBRE: _____

Hojas adicionales adjuntas: _____

[2p] 3. Supón que un sistema de memoria virtual paginada, que utiliza un esquema de traducción directa en un único nivel, tiene un espacio virtual de 256 TiB y un espacio físico de 64 GiB. El tamaño de página es 4 KiB y se dispone de una TLB totalmente asociativa de 512 entradas. El registro base de la tabla de páginas (RBTP) contiene el valor 0x0 0000 0000. Cada entrada de la tabla contiene el bit de residencia (el más significativo), 7 bits de control y el número de página física. El contenido del byte correspondiente a la dirección física X es $(X \bmod 256)$.

(a) Determina el número de bits que ocupa cada campo en una entrada de la tabla de páginas:

(0.1p)

Bit de residencia	Bits de control	Número de página física
1 bit	7 bits	24 bits

(b) Muestra el proceso de traducción de la dirección virtual 0x31F3 1E4B A4CD en su correspondiente dirección física.

- Identifica los campos en los que se descompone la dirección virtual. (0.2p)
- Determina la dirección y el contenido de la entrada correspondiente de la tabla de páginas. (0.3p)
- Comprueba si se podría hacer una traducción válida. (0.1p)
- Tanto si la traducción es válida como si no lo es, calcula la dirección física que resultaría del proceso de traducción. (0.3p)

núm PV	Δ
31F3 1E4B A	4CD
36 bits	12 bits

– Cada entrada de la TP ocupa $1 + 7 + 24 = 32$ bits = 4 Bytes.

$$\&TP[0x31F31E4BA] = RBTP + 0x31F31E4BA \times 4 \text{ B/entrada} = 0xC7CC792E8$$

$$TP[0x31F31E4BA] = E8 \text{ E9 EA EB}$$

Bit Residencia	Bits de control	Número de página física
1	110 1000	E9 EA EB
1 bits	7 bits	24 bits

El bit de residencia vale 1, luego la traducción es válida.

núm PF	Δ
E9 EA EB	4CD
24 bits	12 bits

$$DF = 0xE \text{ 9EAE B4CD}$$

- Escribe el contenido de la entrada en la TLB correspondiente a la traducción anterior. Indica qué cambios se producirían si, justo a continuación, el procesador emitiese la dirección 0x31F3 1E4B A4CE. (0.2p)

La TLB contiene pares {(página virtual) - (página física)} de las últimas traducciones realizadas

núm PV	núm PF
31F31E4BA	E9EAEB
36 bits	24 bits

La dirección 0x31F3 1E4B A4CE se encuentra en la misma página: solo cambian los bits dentro del desplazamiento. Por tanto, la TLB se mantiene igual.

- (c) Calcula el tamaño de la tabla de páginas y compara este tamaño con la memoria disponible en el sistema. ¿Ves algún problema? Si es así, propón una solución. **(0.5p)**

$$|TP| = 2^{36} \text{ entradas} \times (1 + 7 + 24) \text{ bits} = 2^{36} \times 4 \text{ B} = 256 \text{ GiB}$$

El tamaño de la tabla de páginas resulta mayor que la cantidad de memoria física del sistema, lo que es imposible.

Hay diversas soluciones a este problema:

- Paginación en varios niveles
- Reducción de los bits de control, pero todavía no sería suficiente en este caso
- Aumentar la memoria física instalada, que implicaría un aumento del tamaño de cada entrada en la tabla
- Incrementar el tamaño de página
- Reducir el tamaño de la memoria virtual

- (d) Entre los bits de control de cada entrada de la tabla de páginas tenemos un *dirty bit* para indicar si la página en cuestión fue modificada en memoria física o no. Se propone su eliminación para reducir el tamaño de la tabla de páginas. Explica si ves o no algún inconveniente a esta propuesta. **(0.3p)**

El tamaño de cada entrada en la tabla de páginas debe ser un múltiplo de 8 bits para poder ser direccionada (la memoria principal es direccionable a nivel de Byte), por lo que eliminando un único bit no cambiaría el tamaño de la tabla de páginas.

De todas formas, incluso si obviásemos el direccionamiento de la memoria y de ese modo se consiguiese reducir un poco el tamaño de la tabla, ese bit nos permite evitar la escritura en disco de aquellas páginas que son expulsadas de la memoria física y no fueron modificadas. La escritura en disco es una operación muy lenta, por lo que esta medida tendría un impacto muy negativo en el rendimiento del sistema.

APELLIDOS: _____ NOMBRE: _____

Hojas adicionales adjuntas: _____

[0.6p] 4. Tenemos un sistema con memoria virtual de páginas de 16 kiB que conecta la memoria principal con el almacenamiento secundario mediante un bus síncrono a 20 MHz. Las transacciones de las páginas están gestionadas por una DMA a través de este bus en bloques del tamaño de la anchura del bus, necesitando 50 ns para la lectura de cada bloque para la operación de lectura de una página. Tanto la transferencia de un bloque a través del bus como el envío de la dirección al controlador del almacenamiento secundario requieren un ciclo. Las transferencias de datos leídos más recientemente pueden solaparse con la lectura de los siguientes. Sabiendo que la DMA realiza la lectura de una página en una única transacción:

- (a) Calcula la anchura mínima del bus necesaria para que la latencia de la lectura de una página sea menor de 150 μ s. La anchura del bus tiene que ser potencia en base 2 de bytes (2^n bytes, $n \in \mathbb{N}$). (0.4p)

Sendo a anchura do bus 2^n bytes, o número de lecturas para a transacción dunha páxina sería:

$$NL = \frac{16 \text{ kiB}}{2^n \text{ B}} = \frac{2^{14}}{2^n} = 2^{14-n}$$

A frecuencia do bus é de 20 MHz, co cal o período é:

$$T = \frac{1}{20 \times 10^6} = 5 \times 10^{-8} \text{ s} = 50 \text{ ns}$$

A transacción dunha páxina require:

- 1 ciclo para enviar a dirección = 50 ns.
- 50 ns para ler cada un dos bloques. Serían $50 \times 2^{14-n}$ ns.
- 1 ciclo para transferir o último bloque = 50 ns.

Por tanto a latencia para a transacción dunha páxina será:

$$L = 50 + 50 \times 2^{14-n} + 50 = 50 \times (2 + 2^{14-n}) = 0,1 \times (1 + 2^{13-n}) \mu\text{s}$$

Probando varios valores de n podemos ver cal é o mínimo para conseguir unha latencia inferior a 150 μ s:

- $n = 2 : L = 0,1 \times (1 + 2^{11}) = 204,9 \mu\text{s} > 150 \mu\text{s}$
- $n = 3 : L = 0,1 \times (1 + 2^{10}) = 102,5 \mu\text{s} < 150 \mu\text{s}$

Por tanto $n = 3$ e a anchura mínima é 8 bytes (64 bits), cunha latencia de 102,5 μ s.

- (b) Para la anchura del bus determinada, calcula el ancho de banda para las operaciones de lectura de página desde el almacenamiento secundario. (0.2p)

O ancho de banda sería:

$$AB = \frac{16 \text{ kiB}}{102,5 \times 10^{-6} \text{ s}} \approx 16 \times 10^7 \text{ B/s} = 160 \text{ MB/s}$$

[0.4p] 5. Las siguientes afirmaciones contienen algún error. Corrígelas justificando la respuesta.

- (a) Suponiendo discos de 1 TiB y una capacidad neta de almacenamiento de 8 TiB los RAID 5 y RAID 6 tendrán una capacidad de almacenamiento total de 9 TiB, con la información de redundancia repartida cíclicamente a lo largo de todos los discos. (0.2p)

Suponiendo discos de 1 TiB y una capacidad neta de almacenamiento de 8 TiB *el RAID 5 tendrá una capacidad de almacenamiento total de 9 TiB y el RAID 6 de 10 TiB*, con la información de redundancia repartida cíclicamente a lo largo de todos los discos.

Explicación: RAID 6 almacena dos ECCs en discos diferentes, así que necesita un disco más que RAID 5.

- (b) Las desventajas de los discos de estado sólido SSD frente a los discos duros magnéticos HDD son la menor capacidad, el mayor coste y la mayor posibilidad de fallos de funcionamiento. (0.2p)

Las desventajas de los discos de estado sólido SSD frente a los discos duros magnéticos HDD son la menor capacidad y el mayor coste.

Explicación: Los discos SSD son más robustos que los HDD ya que no tienen componentes mecánicos.

APELLIDOS: _____ NOMBRE: _____

- [1p] 1. A continuación se muestra un **fragmento** del diagrama multiciclo que resulta de la ejecución de un código en un procesador MIPS de 5 etapas como el estudiado en clase. En este diagrama no se señalan explícitamente los riesgos ni las anticipaciones. En el código existen instrucciones anteriores y posteriores a este fragmento que desconocemos.

		1	2	3	4	5	6	7	8	9	10	11	12
1	loop: lw \$t0, 0(\$a0)	IF	ID	EX	MEM	WB							
2	lw \$t1, 4(\$a0)		IF	ID	EX	MEM	WB						
3	add \$t2, \$t2, \$t0			IF	ID	–	EX	MEM	WB				
4	addi \$a0, \$a0, 8				IF	–	ID	EX	MEM	WB			
5	sw \$t1, 0(\$a1)						IF	ID	EX	MEM	WB		
6	sw \$t2, 4(\$a1)							IF	ID	EX	MEM	WB	
7	bne \$a1, \$a0, loop								IF	ID	EX	MEM	WB

- (a) [0,4p] Razona (cuando sea posible) si este procesador incorpora las siguientes técnicas estudiadas en clase. En caso afirmativo, indica cuáles de ellas incorpora.
- Unidad de detección de riesgos
 - Unidad de anticipación a la etapa EX
 - Resolución de salto en la etapa ID
 - Salto fijo no efectivo
- (b) [0,3p] Dadas las instrucciones del código anterior, razona si podemos reordenarlas de forma que ocurran riesgos de control, de datos y estructurales, aunque se modifique la lógica original del código.
- (c) [0,3p] Si el procesador implementa la técnica de salto retardado, determina qué instrucciones podemos mover con seguridad al hueco de retardo, indicando también posibles modificaciones necesarias, y razona cuál sería la mejor opción.

- [1p] 2. Un computador dispone de una caché de un único nivel asociativa por conjuntos de 4 vías, de 64 kB en total, con tamaño de línea de 16 bytes.

Considérese el siguiente código:

```
int A[N][M];
for( i = 0; i < M; ++i ) {
    for( j = 0; j < N; ++j ) {
        r += A[j][i];
    }
}
```

Sabiendo que el tamaño de un `int` es de 4 bytes, que las variables escalares se almacenan en registros del micro, y que tanto N como M son potencias de 2, contesta a las siguientes preguntas:

- (a) [0.25p] Calcula el valor de M para que cada acceso al array en una nueva iteración del bucle interno vaya a parar al mismo conjunto caché que en la iteración anterior. Para dicho valor de M, calcula el valor de N de modo que no se produzcan fallos de conflicto en el acceso al array completo.

- (b) [0.25p] Partiendo de los valores calculados de N y M, deduce la relación entre ambos para garantizar un acceso al array libre de fallos de conflicto. Recuerda que tanto N como M se suponen potencias de 2.
- (c) [0.25p] ¿Qué ocurre si duplicamos la asociatividad, pero manteniendo el tamaño total de la caché constante?
- (d) [0.25p] Calcula la tasa de fallos para M=2048, N=16.

3. **Nota:** Este ejercicio no entra dentro del temario del curso 2024/25

- [1p] Considera un computador con un sistema de memoria virtual segmentado. El tamaño del espacio virtual es de 16 TB y el tamaño del espacio físico es de 4 GB. El tamaño máximo de segmento es de 2 GB. En un momento dado, los siguientes segmentos están residentes en memoria física:
- Segmento 0x11, de 256 MB y que comienza en la dirección 0x00000000.
 - Segmento 0x22, de 1 GB y que comienza en la dirección 0x20000000.
 - Segmento 0x33, de 512 MB y que comienza en la dirección 0x80000000.
 - Segmento 0x44, de 256 MB y que comienza en la dirección 0xD0000000.
- (a) [0,2p] Dibuja un esquema que represente el estado de la memoria física del sistema, indicando todos los segmentos residentes.
 - (b) [0,3p] Diseña razonadamente una posible estructura de la tabla de segmentos de este sistema. Ejemplifica su funcionamiento traduciendo la dirección virtual 0x1987654321.
 - (c) [0,3p] Se reciben, secuencialmente, peticiones para ubicar los siguientes segmentos:
 - Segmento 0xAA, de 256 MB.
 - Segmento 0xBB, de 576 MB.
- Explica detalladamente cómo se atenderían con el algoritmo Best-Fit.
- (d) [0,2p] Este sistema de memoria virtual, ¿presenta fragmentación interna? ¿Y fragmentación externa? Justifica tu respuesta apoyándote en los apartados anteriores.

- [1p] 4. Sea un computador con las siguientes características:

- Las direcciones de memoria y las palabras son de 32 bits.
- Tiene un sistema de memoria que permite accesos en bloques de 4, 8 y 16 palabras.
- Tenemos dos buses candidatos para conectarse a este sistema de memoria, ambos son síncronos, de 32 bits y tienen la misma frecuencia de reloj. El bus A está configurado para soportar accesos a memoria en bloques de 8 palabras de 32 bits y no requiere de ningún ciclo de espera entre dos operaciones de bus, es decir, entre el envío de dos bloques de palabras no hay espera alguna. En cambio, el bus B solo soporta accesos a memoria en bloques de 16 palabras de 32 bits y además necesita esperar un número de ciclos entre el envío de dos bloques de palabras (se supondrá el bus libre antes de cada acceso).
- Para cada bloque, la memoria tarda lo equivalente a 20 ciclos del bus en acceder a las primeras 4 palabras del bloque; y luego cada grupo adicional de cuatro palabras del mismo bloque se lee en 10 ciclos de bus. Se considerará que la transferencia de los datos leídos más recientemente por el bus puede solaparse con la lectura de las cuatro palabras siguientes.

Calcular:

- (a) **[0.65p]** El número de ciclos de espera que tendrá que realizar el Bus B en cada transacción para que el ancho de banda del Bus A sea igual al ancho de banda del Bus B cuando se leen 128 palabras de memoria.
- (b) **[0.35p]** Este computador tiene un total de 1024 GB de información almacenada en 4 discos de 256 GB en RAID 0. Me dispongo a montar un sistema RAID para tener cierta de tolerancia a fallos, pero que me ofrezca concurrencia a la hora de acceder a los datos. ¿Qué configuración RAID escogería y cuántos discos requeriría? Hay una oferta de un proveedor que me ofrece a precio muy económico hasta 8 discos de la misma serie de fabricación. ¿Es buena idea?

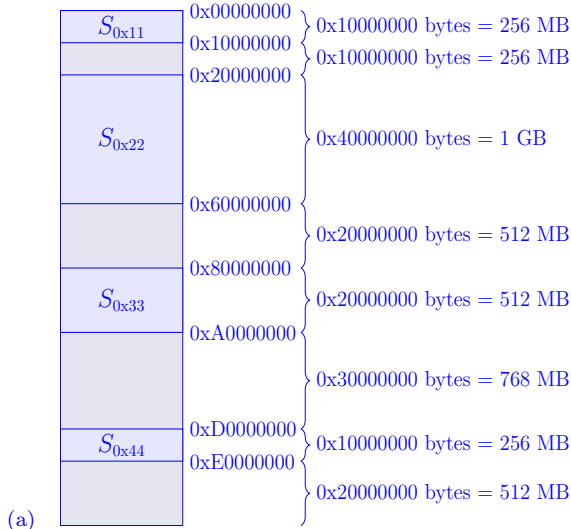
1. —————

- (a) Hay unidad de detección de riesgos, porque de lo contrario no podríamos tener la burbuja en la instrucción 3. No hay unidad de anticipación a la etapa EX, porque entonces el riesgo anterior no se habría producido al poder anticipar \$t_0\$ a la instrucción 3. La resolución de salto en la etapa ID y salto fijo no efectivo no lo podemos saber porque cuando el salto ejecuta su etapa ID los datos que necesita ya están actualizados en el banco de registros y por tanto no sabemos si la condición de salto se evalúa en ID o en alguna etapa posterior, y además en el diagrama no se muestra qué ocurre después del salto.
- (b) El código original ya presenta un riesgo de datos, en concreto un riesgo RAW entre las instrucciones 1 y 3. La instrucción 6 conlleva un riesgo de control tras su ejecución salvo que implemente salto retardado, tanto si se implementa una predicción de salto efectiva como no efectiva. Con las instrucciones disponibles no es posible forzar un riesgo estructural, porque todas tienen la misma latencia y por lo tanto nunca coincidirán en la misma etapa.
- (c) Las instrucciones 1, 2 y 4 se podrían mover “desde destino de salto” con cierta modificación, pero eso implicaría que se ejecuten una vez más al finalizar el bucle. Como desconocemos la estructura de los datos reservados en memoria y el código que se ejecuta tras el bucle, no es un movimiento seguro. La instrucción 3 no puede moverse por dependencias con instrucciones anteriores y posteriores. Las instrucciones 5 y 6 pueden moverse con seguridad puesto que no tienen dependencias con las instrucciones posteriores. Si el salto se decide en ID, cosa que no sabemos, se crearía un riesgo entre las instrucciones 4 y 7 (al no poder anticipar el valor de \$a_0\$) y no se obtendría ningún beneficio.

2. —————

- (a) $T_{cache} = 2^{16}B, T_{linea} = 2^4B, \#vias = 2^2$: Hay $2^{16}/2^4/2^2 = 2^{10}$ conjuntos. En cada iteración saltamos M elementos de 2^2 Bytes: $2 \times M$ Bytes. Para coincidir de nuevo en el mismo conjunto, el salto debe ser de 2^{10} conjuntos $\times 2^4$ Bytes/linea = 2^{14} Bytes. Como cada elemento de A ocupa 2^2 Bytes $\rightarrow M = 2^{12} = 4096$.
Como hay 4 vías, podemos repetir el mismo conjunto 4 veces sin que ocurra un fallo de conflicto, por lo que $N \leq 4$.
- (b) Partiendo de los valores anteriores, si se reduce M a la mitad se alternarán el uso de 2 conjuntos (8 vías). Si se vuelve a reducir a la mitad, 4 conjuntos (16 vías), etc. Por lo tanto al reducir M podemos incrementar N proporcionalmente manteniendo la tasa de fallos. Entonces, la relación entre M y N debe ser $M = 2^k, N \leq \max(4, 2^{14-k})$. Es decir, si M pasa de 4096 no afecta a N , que debe mantenerse en un valor menor o igual que la asociatividad.
- (c) Se permiten valores mayores de N , en concreto $N \leq \max(8, 2^{13-k})$.
- (d) Dado que los valores ($M = 2^{11}, N = 2^4$) no cumplen lo calculado anteriormente, la tasa de fallos será del 100%. Como explicación adicional, con $M = 2^{11}$ se repite el mismo conjunto cada 2 iteraciones. En las 8 primeras iteraciones del bucle interno se llenarán los 2 conjuntos accedidos completamente, y en las 8 iteraciones restantes se sobrescribirán los datos de la caché, y de esa forma no podrán ser reutilizados en el acceso a la segunda columna de la matriz A .

3. —————



(b) La estructura de cada entrada podría ser:

Residencia (1)	Otros bits de control (c)	Dirección base (32)	Tamaño (31)
----------------	---------------------------	---------------------	-------------

La dirección base es una dirección física completa: $|F| = 4 \text{ GB} = 2^{32} B$

La entrada “Tamaño” está condicionado por el tamaño máximo de segmento: $\max|S| = 2 \text{ GB} = 2^{31} B$

Deberíamos escoger un número de bits c tal que el tamaño total de la entrada fuese un número entero de bytes (es decir, $c = 0, 8, 16 \dots$).

$|V| = 16 \text{ TB} = 2^{44} B \Rightarrow \text{DV de 44 bits}$

nSV	Δ_s
13 bits	31 bits

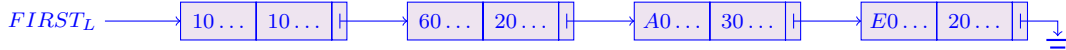
Dividiendo los bits de 0x1987654321 tenemos que nSV = 0x0033 y $\Delta_s = 0x07654321$. El SV 0x33 es uno de los segmentos que mencionan en el enunciado como residentes. La entrada correspondiente en la tabla sería:

Residencia (1)	Otros bits de control (c)	Dirección base (32)	Tamaño (31)
1	—	0x80000000	0x20000000

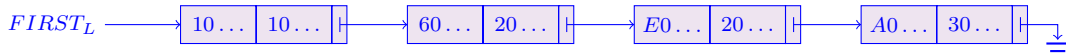
Tenemos que comprobar también si el desplazamiento está dentro del tamaño del segmento: $\Delta_s = 0x07654321 \leq 0x20000000$.

Con lo que la dirección física a la que se traduce será $\text{DF} = 0x80000000 + 0x07654321 = 0x87654321$

(c) La lista enlazada del sistema sería:



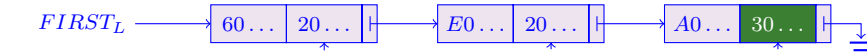
El algoritmo Best-Fit busca el espacio libre más pequeño con el tamaño suficiente. Por ello, trabaja con la lista ordenada por tamaño creciente de espacio libre.



El segmento 0xAA tiene tamaño $|SV_{0xAA}| = 256 \text{ MB} = 2^{28} B = 0x10000000 B$, por lo que se asigna al primer espacio libre de la lista. La lista queda como:



El segmento 0xBB tiene tamaño $|SV_{0xBB}| = 576 \text{ MB} = 0x24000000 B$, por lo que se asigna al último espacio libre de la lista. La lista queda como:



$$\begin{aligned} \text{Base}(SV_{0xBB}) &= 0xA0000000 + 0x30000000 - 0x24000000 \\ \text{Base}(SV_{0xBB}) &= 0xAC000000 \\ \text{Tamaño}(Q) &= 0x30000000 - 0x24000000 \\ \text{Tamaño}(Q) &= 0x0C000000 \end{aligned}$$

(d) No presenta fragmentación interna: los segmentos virtuales tienen tamaño variable para ajustarse a los datos/código que sea necesario.

En cambio, sí que presenta fragmentación externa: es común que vayan quedando espacios libres en memoria de tamaño tan pequeño que no sean útiles. Un ejemplo de esta situación sería el nuevo nodo Q de 0x0C000000 bytes que se ha generado en el apartado anterior.

(a) BUS A:

Son $128/8 = 16$ transacciones donde cada una tiene 1 ciclo para el envío de la dirección + 20 ciclos para las primeras cuatro palabras + $\max(4 \text{ ciclos}, 10 \text{ ciclos}) + 4$ ciclos para enviar las últimas 4 palabras = 35 ciclos. En 16 transacciones son 560 ciclos.

BUS B:

Son $128/16 = 8$ transacciones donde cada una tiene 1 ciclo env. + 20c primeras cuatro + $\max(4c, 10c) + \max(4c, 10c) + \max(4c, 10c) + 4c$ env. últ. 4 pal + Tespera = $55 + Tespera$ ciclos. En 8 transacciones son $440 + 8 Tespera$ ciclos.

Para que $AB_A == AB_B$ entonces el número ciclos de Bus A tiene que ser igual que el del Bus B:

$560 = (55 + T_{esp}) \cdot 8$ y por tanto $T_{esp} = 15$ ciclos.

(b) RAID 4, 5 y 6 son la mejor opción donde se necesitan uno, uno y dos discos extras respectivamente. Sin embargo, RAID 4 forma un cuello de botella al no tener la redundancia distribuída por lo que RAID 5 y RAID 6 son mejores opciones. Para presentar mayor tasa de tolerancia a fallos, es mejor quedarnos con RAID 6 al tener hasta 2 discos de recuperación.

No es buena idea porque empíricamente se demuestra que es más probable que de haber un fallo en un disco, todos los de su serie de fabricación lo contengan igualmente, lo que nos reduce la vida esperada del sistema RAID.